数字电路

Digital Circuits · 学习笔记

作者：魏舟

日期：2026 年 10 月 18 日

Stay hungry. Stay foolish.

求知若饥，虚心若愚。

—— 史蒂夫·乔布斯（Steve Jobs），2005

前言

本笔记按课程内容整理，包括第一部分「数字电路基础」（第一章 数制与 BCD 码、第二章 布尔代数、第三章 组合逻辑电路基础）与第二部分「时序逻辑电路」（第四章 时序逻辑电路引论、第五章 时序逻辑电路的分析与设计）。

整理原则：概念与公式沿用课件原文表述，仅作版式重排；课件中被提取工具打散的公式已按标准写法还原。

目录

前言 …… ii

目录 …… iii

第一部分 数字电路基础

第一章 数制与 BCD 码 …… 2

1.1 常用数制 …… 2

1.1.1 常用数制一览 …… 2

1.1.2 二进制与十进制之间的转换 …… 3

1.1.3 二进制算术运算 …… 3

1.1.4 带符号数的表示：原码、反码和补码 …… 4

1.2 几种简单的编码 …… 6

1.2.1 二-十进制码（BCD 码） …… 6

1.2.2 格雷码（Gray 码） …… 7

1.2.3 计算机字符集 …… 7

第二章 布尔代数 …… 8

2.1 基本逻辑运算 …… 8

2.1.1 与逻辑运算 …… 8

2.1.2 或逻辑运算 …… 9

2.1.3 非逻辑运算 …… 10

2.2 复合逻辑运算 …… 10

2.2.1 与非逻辑 …… 10

2.2.2 或非逻辑 …… 11

2.2.3 与或非逻辑 ····· 11
2.2.4 异或逻辑 ····· 13
2.2.5 同或逻辑 ····· 13
2.3 逻辑电平及正、负逻辑 ····· 14
2.4 基本定律和规则 ····· 14
2.4.1 逻辑函数的相等 ····· 14
2.4.2 基本定律 ····· 15
2.4.3 逻辑代数的三条规则 ····· 15
2.5 常用公式 ····· 16
2.5.1 消去律 ····· 16
2.5.2 吸收律 1 ····· 16
2.5.3 吸收律 2 ····· 17
2.5.4 包含律 ····· 17
2.5.5 关于异或和同或运算 ····· 18
2.6 逻辑函数的标准形式 ····· 18
2.6.1 函数的“与–或”式和“或–与”式 ····· 18
2.6.2 最小项 ····· 19
2.6.3 最大项 ····· 20
2.6.4 最小项和最大项的关系 ····· 20
2.6.5 逻辑函数的最小项之和形式 ····· 21
2.6.6 逻辑函数的最大项之积形式 ····· 21
2.6.7 两种标准形式之间的关系 ····· 22
2.7 真值表与逻辑表达式 ····· 22
2.7.1 由逻辑函数式列真值表 ····· 22
2.7.2 由真值表写逻辑函数式 ····· 23
2.8 逻辑函数的化简 ····· 23
2.8.1 公式法 ····· 23
2.8.2 卡诺图法 ····· 24

第三章 组合逻辑电路基础 ····· 29
3.1 由基本逻辑门构成的组合电路的设计与分析 ····· 29
3.1.1 一般分析方法 ····· 29
3.1.2 一般设计方法 ····· 30
3.1.3 自顶向下的模块化设计方法 ····· 32
3.2 MSI 构成的组合逻辑电路 ····· 34
3.2.1 编码器 ····· 34
3.2.2 译码器 ····· 40
3.2.3 数据选择器 ····· 46
3.2.4 加法器 ····· 49
3.2.5 数值比较器 ····· 53

第二部分 时序逻辑电路

第四章 时序逻辑电路引论 ····· 56
4.1 时序逻辑电路的基本概念 ····· 56
4.1.1 结构模型 ····· 56
4.1.2 状态表和状态图 ····· 57
4.2 存储器件 ····· 58
4.3 锁存器 ····· 59
4.3.1 RS 锁存器 ····· 59
4.3.2 门控 RS 锁存器 ····· 60
4.3.3 D 锁存器 ····· 60
4.4 触发器 ····· 61
4.4.1 主从触发器 ····· 61
4.4.2 边沿触发器 ····· 63
4.4.3 集成触发器 ····· 64
4.5 触发器逻辑功能的转换 ····· 64
4.5.1 代数法 ····· 64

4.5.2 图表法 ····· 65
4.5.3 多功能触发器 ····· 66
4.6 触发器应用举例 ····· 66
4.6.1 消颤开关 ····· 66
4.6.2 单脉冲发生器 ····· 67
第五章 时序逻辑电路的分析与设计 ····· 68
5.1 MSI 构成的时序逻辑电路 ····· 68
5.1.1 寄存器和移位寄存器 ····· 68
5.1.2 计数器 ····· 70
5.1.3 移位寄存器型计数器 ····· 76
5.1.4 序列信号发生器 ····· 77
5.2 同步时序逻辑电路的分析方法 ····· 79
5.3 同步时序逻辑电路的设计 ····· 81

第一部分 数字电路基础

第一章 数制与 BCD 码

数制（进位计数制）：用进位的方法来计数，包括**计数符号（数码）**和**进位规则**两个方面。常用的有十进制、二进制、八进制、十六进制等。

1.1 常用数制

1.1.1 常用数制一览

数制	计数符号	进位规则	按权展开式
十进制	$0 \sim 9$	逢十进一	$(N)_{10}=\sum_{i=-m}^{n-1} a_i \times 10^i$
二进制	$0, 1$	逢二进一	$(N)_{2}=\sum_{i=-m}^{n-1} a_i \times 2^i$
八进制	$0 \sim 7$	逢八进一	$(N)_{8}=\sum_{i=-m}^{n-1} a_i \times 8^i$
十六进制	$0 \sim 9$、$A \sim F$	逢十六进一	$(N)_{16}=\sum_{i=-m}^{n-1} a_i \times 16^i$

表 1.1　常用数制一览

其中 a_i 为系数、R^i 为权（R 为基数）。

例：

$$\begin{aligned} 1987.45 = & 1\times 10^3+9\times 10^2+8\times 10^1+7\times 10^0 \\ & +4\times 10^{-1}+5\times 10^{-2} \end{aligned} \tag{1.1}$$

$$(6\mathrm{D}.4\mathrm{B})_{16}=6\times 16^1+13\times 16^0+4\times 16^{-1}+11\times 16^{-2} \quad (\mathrm{D}=13, \mathrm{B}=11) \tag{1.2}$$

$$(63.45)_8=6\times 8^1+3\times 8^0+4\times 8^{-1}+5\times 8^{-2} \tag{1.3}$$

数字电路中采用二进制的原因：

1. 数字装置简单可靠；
2. 二进制数运算规则简单；
3. 数字电路既可以进行算术运算，也可以进行逻辑运算。

1.1.2 二进制与十进制之间的转换

（1）二进制数转换为十进制数（按权展开法）

$$\begin{aligned}(1011.101)_2 &= 1\times 2^3+0\times 2^2+1\times 2^1+1\times 2^0 \\ &\quad +1\times 2^{-1}+0\times 2^{-2}+1\times 2^{-3} \\ &= 8+2+1+0.5+0.125=(11.625)_{10}\end{aligned} \tag{1.4}$$

（2）十进制数转换为二进制数（提取 2 的幂法）

$$\begin{aligned}(45.5)_{10} &= 32+8+4+1+0.5 \\ &= 1\times 2^5+0\times 2^4+1\times 2^3+1\times 2^2+0\times 2^1+1\times 2^0+1\times 2^{-1} \\ &= (101101.1)_2\end{aligned} \tag{1.5}$$

数制转换还可以采用**基数连除、连乘**（除基数取余、乘基数取整）等方法。

1.1.3 二进制算术运算

算术运算规则与十进制相同，区别是**逢二进一**。其特点是：加、减、乘、除全部可以用**移位**和**相加**两种操作实现，电路结构简化，所以数字电路中普遍采用二进制算术运算。

加法：逐位相加并处理进位。

$$S_0=A_0+B_0(\text{进位 } C_1),\quad S_1=A_1+B_1+C_1(\text{进位 } C_2),\quad \cdots \tag{1.6}$$

设 $A=A_3A_2A_1A_0$、$B=B_3B_2B_1B_0$，两数之和为 $S=S_3S_2S_1S_0$，则

$$A+B=C_4S_3S_2S_1S_0 \tag{1.7}$$

减法：将减数 B 变成补码后与被减数 A 相加，若有进位则舍去，其和即为两数之差。例：

$$\begin{aligned}Y &= (1000)_2-(0100)_2=(1000)_2+[(1011)_2+1] \\ &= (1000)_2+(1100)_2=(10100)_2\xrightarrow{\text{舍去进位}}(0100)_2\end{aligned} \tag{1.8}$$

乘法：乘以 2 相当于**左移一位**。方法有：（1）按十进制的乘法过程；（2）采用移位相加的方法。

除法：除以 2^k 相当于**右移 k 位**。例（$k=2$，即除以 4）：

$$(00001011)_2\div(100)_2=(00000010)_2(\text{商}),\quad \text{余数}=(11)_2 \tag{1.9}$$

1.1.4 带符号数的表示：原码、反码和补码

计算机中的带符号数表示为**符号位 + 真值**（合称**机器数**），符号位用“0”表示正、“1”表示负。

定义 1.1 (原码)

真值 X 的原码记为 $[X]_{原}$：最高位为符号位，其余为真值部分；不论数的正负，数值部分均保持原真值不变。

优点：正负号与原真值之间的对应关系简单，容易理解；

缺点：用原码进行加减运算比较困难，0 的表示不唯一：$+0 = 0\,0000000$，$-0 = 1\,0000000$。

定义 1.2 (反码)

真值 X 的反码记为 $[X]_{反}$：若 $X > 0$，则 $[X]_{反} = [X]_{原}$；若 $X < 0$，则符号位不变、数值部分按位取反。0 的反码也不唯一：$[+0]_{反} = 00000000$，$[-0]_{反} = 11111111$。

定义 1.3 (补码)

真值 X 的补码记为 $[X]_{补}$：若 $X > 0$，则 $[X]_{补} = [X]_{反} = [X]_{原}$；若 $X < 0$，则

$$[X]_{补} = [X]_{反} + 1 \tag{1.10}$$

补码的符号位与 $[X]_{原}$ 相同，0 的表示唯一，且可将减法运算转换为加法运算。

例（$X = -52 = -0110100$）：

$$[X]_{原} = 1\,0110100, \quad [X]_{反} = 1\,1001011, \quad [X]_{补} = [X]_{反} + 1 = 11001100 \tag{1.11}$$

0 的编码：8 位原码中 $+0 = 0\,0000000$、$-0 = 1\,0000000$，不唯一；补码表示唯一：

$$[-0]_{补} = [-0]_{反} + 1 = 11111111 + 1 = 1\,00000000 \xrightarrow{8\text{ 位字长舍去进位}} 00000000 \tag{1.12}$$

特殊数 10000000：

- 原码中定义为 -0；
- 反码中定义为 -127；
- 补码中定义为 -128；
- 作为无符号数时 $(10000000)_2 = 128$。

表示范围（8 位）：

编码	表示范围
原码 / 反码	$-127 \sim +127$
补码	$-128 \sim +127$
无符号数	$0 \le X \le 2^n - 1$

表 1.2　8 位数的表示范围

补码运算：通过引进补码，可将减法运算转换为加法运算，即

$$[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}, \quad [X-Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}} \tag{1.13}$$

例子 1.1 (补码加法)

$X = -0110100$，$Y = +1110100$，求 $X+Y$。

解.

$$[X]_{\text{补}} = 11001100, \quad [Y]_{\text{补}} = 01110100 \tag{1.14}$$

所以

$$[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}} = 11001100 + 01110100 = 01000000 \tag{1.15}$$

符号位讨论——用二进制补码求 13 ± 10、-13 ± 10（6 位字长）：

$$\begin{aligned} 13 + 10 &= 0\,01101 + 0\,01010 = 0\,10111 = 23, \\ 13 - 10 &= 0\,01101 + 1\,10110 = 0\,00011 = 3, \\ -13 + 10 &= 1\,10011 + 0\,01010 = 1\,11101 = -3, \\ -13 - 10 &= 1\,10011 + 1\,10110 = 1\,01001 = -23 \end{aligned} \tag{1.16}$$

无符号数的溢出：无符号数的范围为 $0 \le X \le 2^n - 1$，若运算结果超出这个范围，则产生**溢出**。溢出判断准则：运算时，当最高位向更高位有进位（或借位）时则产生溢出。

1.2 几种简单的编码

1.2.1 二-十进制码（BCD 码）

定义 1.4 (二-十进制码（BCD 码）)

用四位二进制代码来表示一位十进制数码，这样的代码称为二-十进制码（BCD 码，Binary Coded Decimal codes）。四位二进制有 16 种不同组合，可以在这 16 种代码中任选 10 种表示十进制数的 10 个不同符号，选择方法不同，就得到不同的编码形式。常见的有 8421 码、5421 码、2421 码、余 3 码等。

十进制数	8421 码	5421 码	2421 码	余 3 码
0	0000	0000	0000	0011
1	0001	0001	0001	0100
2	0010	0010	0010	0101
3	0011	0011	0011	0110
4	0100	0100	0100	0111
5	0101	1000	1011	1000
6	0110	1001	1100	1001
7	0111	1010	1101	1010
8	1000	1011	1110	1011
9	1001	1100	1111	1100

表 1.3 常用 BCD 码

定义 1.5 (有权 BCD 码)

每位数码都有确定的位权的码，例如 8421 码、5421 码、2421 码。

例：5421 码 1011 代表 $5+0+2+1=8$；2421 码 1100 代表 $2+4+0+0=6$。

5421 码与 2421 码的编码不唯一，例如 2421 码 0110 也可表示 6。

由表 1.3 可见：

1. 8421 码与代表 0 ～ 9 的二进制数一一对应；
2. 5421 码前 5 个码与 8421 码相同，后 5 个码在前 5 个码的基础上加 1000 构成；

3. 2421 码前 5 个码与 8421 码相同，后 5 个码以中心对称取反，这样的码称为**自反代码**。

例如 $4 \to 0100$、$5 \to 1011$；$0 \to 0000$、$9 \to 1111$。

定义 1.6 (无权 BCD 码（余 3 码）)

每位数码无确定的位权，例如余 3 码。余 3 码的编码规律为：在 8421 码上加 0011。

例如 6 的余 3 码为 $0110 + 0011 = 1001$。

1.2.2 格雷码（Gray 码）

格雷码为无权码，特点为：相邻两个代码之间仅有一位不同，其余各位均相同，具有这种特点的代码称为**循环码**，格雷码是循环码。

格雷码与四位二进制码之间的关系：设四位二进制码为 $B_3B_2B_1B_0$，格雷码为 $R_3R_2R_1R_0$，则

$$R_3 = B_3, \quad R_2 = B_3 \oplus B_2, \quad R_1 = B_2 \oplus B_1, \quad R_0 = B_1 \oplus B_0 \tag{1.17}$$

其中 $\oplus$ 为异或运算符，运算规则：两运算数相同结果为“0”，不同结果为“1”。

1.2.3 计算机字符集

各种文字和符号的总称，包括各国家文字、标点符号、图形符号、数字等。常见字符集：ASCII、GB2312、BIG5、GB18030、Unicode。

第二章 布尔代数

数字电路的基础为逻辑代数，由英国数学家 George Boole 在 1847 年提出，逻辑代数也称布尔代数。

2.1 基本逻辑运算

在逻辑代数中，变量常用字母 $A, B, C, \cdots, Y, Z$、$a, b, c, \cdots, x, y, z$ 等表示，变量的取值只能是“0”或“1”。

逻辑代数中只有三种基本逻辑运算，即“与”、“或”、“非”。

2.1.1 与逻辑运算

定义 2.1 (与逻辑运算)

只有决定一事件的**全部条件**都具备时，这件事才成立；如果有一个或一个以上条件不具备，则这件事就不成立。这样的因果关系称为“与”逻辑关系，其函数式为 $F = A \cdot B = AB$。

与门逻辑功能概括：有“0”出“0”，全“1”出“1”。

与门逻辑符号如下：

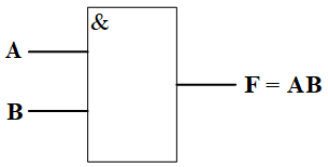


图 2.1　与门逻辑符号

与逻辑真值表如下：

A	B	$F=AB$
0	0	0
0	1	0
1	0	0
1	1	1

表 2.1 与逻辑真值表

2.1.2 或逻辑运算

定义 2.2 (或逻辑运算)

在决定一事件的各种条件中，只要有一个或一个以上条件具备时，这件事就成立；**只有**所有的条件**都不具备**时，这件事才**不成立**。这样的因果关系称为“或”逻辑关系，其函数式为 $F=A+B$。

或门逻辑功能概括：有“1”出“1”，全“0”出“0”。

或门逻辑符号如下：

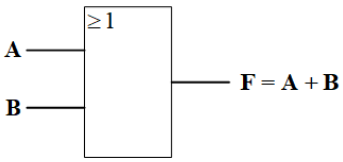


图 2.2 或门逻辑符号

或逻辑真值表如下：

A	B	$F=A+B$
0	0	0
0	1	1
1	0	1
1	1	1

表 2.2 或逻辑真值表

注意

与门和或门均可以有多个输入端。

2.1.3 非逻辑运算

定义 2.3 (非逻辑运算)

假定事件 F 成立与否同条件 A 的具备与否有关：若 A 具备，则 F 不成立；若 A 不具备，则 F 成立。F 和 A 之间的这种因果关系称为“非”逻辑关系，其函数式为 $F = \overline{A}$。

非门逻辑符号如下：

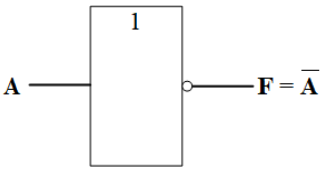


图 2.3 非门逻辑符号

非逻辑真值表如下：

A	$F = \overline{A}$
0	1
1	0

表 2.3 非逻辑真值表

2.2 复合逻辑运算

2.2.1 与非逻辑

与非逻辑由与逻辑和非逻辑组合而成，函数式为 $F = \overline{A \cdot B} = \overline{AB}$。

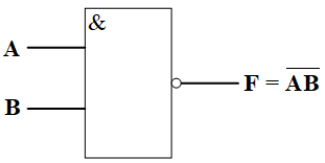


图 2.4 与非门逻辑符号

与非逻辑真值表如下：

A	B	$F=\overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

表 2.4 与非逻辑真值表

2.2.2 或非逻辑

或非逻辑由或逻辑和非逻辑组合而成，函数式为 $F=\overline{A+B}$。

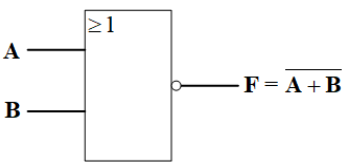


图 2.5 或非门逻辑符号

或非逻辑真值表如下：

A	B	$F=\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

表 2.5 或非逻辑真值表

2.2.3 与或非逻辑

与或非逻辑由与、或、非三种逻辑组合而成，函数式为 $F=\overline{AB+CD}$。

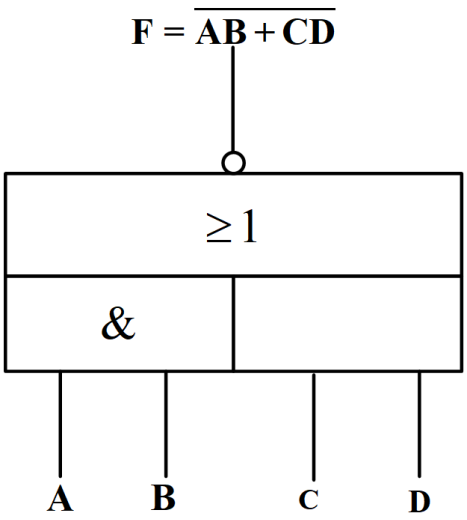


图 2.6 与或非门逻辑符号

与或非逻辑真值表如下：

A	B	C	D	$F=\overline{AB+CD}$
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

表 2.6 与或非逻辑真值表

2.2.4 异或逻辑

异或逻辑的函数式为 $F = \overline{A}B + A\overline{B} = A \oplus B$，口诀：相同得“0”，相异得“1”。

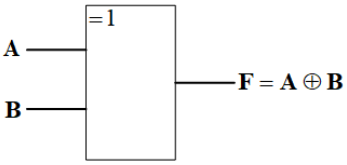


图 2.7 异或门逻辑符号

异或逻辑真值表如下：

A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

表 2.7 异或逻辑真值表

2.2.5 同或逻辑

同或逻辑的函数式为 $F = \overline{AB} + AB = A \odot B$，口诀：相同得“1”，相异得“0”。

同或和异或互为反函数，即

$$A \oplus B = \overline{A \odot B} \tag{2.1}$$

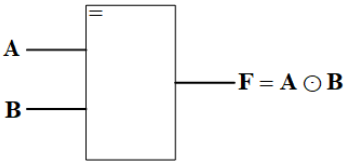


图 2.8 同或门逻辑符号

同或逻辑真值表如下：

A	B	$F=A\odot B$
0	0	1
0	1	0
1	0	0
1	1	1

表 2.8 同或逻辑真值表

2.3 逻辑电平及正、负逻辑

门电路的输入、输出为二值信号，用“0”和“1”表示，这里的“0”、“1”一般用两个不同电平值来表示。

定义 2.4 (正逻辑与负逻辑)

- **正逻辑**：若用高电平 V_H 表示逻辑“1”，用低电平 V_L 表示逻辑“0”，则称为正逻辑约定，简称正逻辑；
- **负逻辑**：若用高电平 V_H 表示逻辑“0”，用低电平 V_L 表示逻辑“1”，则称为负逻辑约定，简称负逻辑。

一般采取正逻辑表示。

注意

对一个特定的逻辑门，采用不同的逻辑表示时，其门的名称也就不同。

2.4 基本定律和规则

2.4.1 逻辑函数的相等

设有两个逻辑函数

$$F_1=f_1(A_1,A_2,\cdots,A_n),\quad F_2=f_2(A_1,A_2,\cdots,A_n) \tag{2.2}$$

如果对于 $A_1,A_2,\cdots,A_n$ 的任何一组取值（共 2^n 组），F_1 和 F_2 均相等，则称 F_1 和 F_2 相等。

因此，如两个函数的真值表相等，则这两个函数一定相等。

2.4.2 基本定律

定律	与形式	或形式
① 0-1 律	$A \cdot 0 = 0$	$A + 1 = 1$
② 自等律	$A \cdot 1 = A$	$A + 0 = A$
③ 重叠律	$A \cdot A = A$	$A + A = A$
④ 互补律	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
⑤ 交换律	$A \cdot B = B \cdot A$	$A + B = B + A$
⑥ 结合律	$A(BC) = (AB)C$	$A + (B + C) = (A + B) + C$
⑦ 分配律	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
⑧ 反演律	$\overline{A + B} = \overline{A} \cdot \overline{B}$	$\overline{AB} = \overline{A} + \overline{B}$

表 2.9 逻辑代数的基本定律

反演律也称**德·摩根定理**，是一个非常有用的定理。

2.4.3 逻辑代数的三条规则

2.4.3.1 代入规则

任何一个含有变量 x 的等式，如果将所有出现 x 的位置，都用一个逻辑函数式 F 代替，则等式仍然成立。

2.4.3.2 反演规则

设 F 为任意逻辑表达式，若将 F 中所有运算符、常量及变量作如下变换：

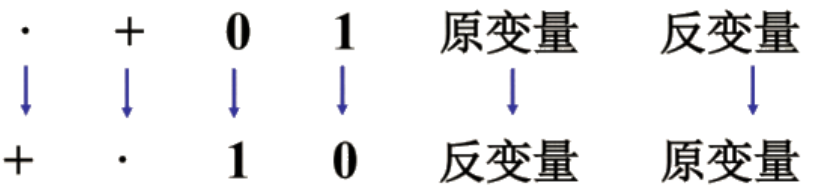


图 2.9 反函数的变换规则

则所得新的逻辑式即为 F 的反函数，记为 $\overline{F}$。

注意：

1. 保持原式运算的优先次序；
2. 原式中的不属于单变量上的非号不变。

2.4.3.3 对偶规则

设 F 为任意逻辑表达式，若将 F 中所有运算符和常量作如下变换：

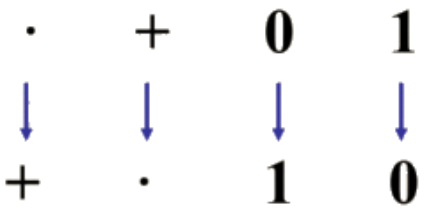


图 2.10 对偶式的变换规则

则所得新的逻辑表达式即为 F 的对偶式，记为 F'。

2.5 常用公式

2.5.1 消去律

定理 2.1 (消去律)

$$AB + A\overline{B} = A \tag{2.3}$$

证明.

$$\begin{aligned} AB + A\overline{B} = A\left(B + \overline{B}\right) = A \cdot 1 = A \\ \xrightarrow{\text{对偶关系}} (A + B)\left(A + \overline{B}\right) = A \end{aligned} \tag{2.4}$$

□

2.5.2 吸收律 1

定理 2.2 (吸收律 1)

$$A + AB = A \tag{2.5}$$

证明.

$$\begin{aligned} A + AB &= A(1 + B) = A \cdot 1 = A \\ &\xrightarrow{\text{对偶关系}} A(A + B) = A \end{aligned} \tag{2.6}$$

□

2.5.3 吸收律 2

定理 2.3 (吸收律 2)

$$A + \overline{A}B = A + B \tag{2.7}$$

证明.

$$\begin{aligned} A + \overline{A}B &= \left(A + \overline{A}\right)(A + B) = 1 \cdot (A + B) \\ &\xrightarrow{\text{对偶关系}} A\left(\overline{A} + B\right) = AB \\ &= A + B \end{aligned} \tag{2.8}$$

□

2.5.4 包含律

定理 2.4 (包含律)

$$AB + \overline{A}C + BC = AB + \overline{A}C \tag{2.9}$$

证明.

$$\begin{aligned} AB + \overline{A}C + BC &= AB + \overline{A}C + \left(A + \overline{A}\right)BC \\ &= AB + \overline{A}C + ABC + \overline{A}BC \\ &= AB(1 + C) + \overline{A}C(1 + B) \end{aligned} \tag{2.10}$$

由对偶关系，可得

$$(A + B)\left(\overline{A} + C\right)(B + C) = (A + B)\left(\overline{A} + C\right) \tag{2.11}$$

□

2.5.5 关于异或和同或运算

性质 2.1 (对偶数个变量)

$$A_1 \oplus A_2 \oplus \cdots \oplus A_n = \overline{A_1 \odot A_2 \odot \cdots \odot A_n} \tag{2.12}$$

性质 2.2 (对奇数个变量)

$$A_1 \oplus A_2 \oplus \cdots \oplus A_n = A_1 \odot A_2 \odot \cdots \odot A_n \tag{2.13}$$

异或和同或的其他性质见表 2.10。

异或	同或
$A \oplus 0 = A$	$A \odot 1 = A$
$A \oplus 1 = \overline{A}$	$A \odot 0 = \overline{A}$
$A \oplus A = 0$	$A \odot A = 1$
$A \oplus (B \oplus C) = (A \oplus B) \oplus C$	$A \odot (B \odot C) = (A \odot B) \odot C$
$A(B \oplus C) = AB \oplus AC$	$A + (B \odot C) = (A + B) \odot (A + C)$

表 2.10 异或和同或的其他性质

2.6 逻辑函数的标准形式

2.6.1 函数的“与–或”式和“或–与”式

定义 2.5 (“与--或”式)

“与–或”式指一个函数表达式中包含若干个“与”项，这些“与”项的“或”表示这个函数，如

$$F(A, B, C, D) = A + B\overline{C} + ABC\overline{D} \tag{2.14}$$

定义 2.6 (“或--与”式)

“或–与”式指一个函数表达式中包含若干个“或”项，这些“或”项的“与”表示这个函数，如

$$F(A, B, C, D) = (A + C + D)\left(\overline{B} + D\right)\left(A + \overline{B} + \overline{D}\right) \tag{2.15}$$

2.6.2 最小项

定义 2.7 (最小项)

最小项是“与”项，具有以下特点：

1. n 个变量构成的每个最小项一定包含 n 个因子的乘积项；
2. 每个变量必须以原变量或反变量形式作为因子出现**一次且仅一次**。

例如：两变量有 $2^2=4$ 个最小项：$A\overline{B}$、$\overline{A}B$、$\overline{A}\overline{B}$、AB；三变量有 $2^3=8$ 个最小项。

定义 2.8 (最小项的编号)

任一最小项用 m_i 表示，m 表示最小项，下标 i 为使该最小项为 1 的变量取值所对应的等效十进制数。

例如：使最小项 $A\overline{B}C$ 为 1 的取值 $A,B,C=1,0,1$，二进制数 101 等效十进制数为 5，所以 $A\overline{B}C=m_5$。

性质 2.3 (最小项的性质)

1. 变量任取一组值，仅有一个最小项为 1，其他最小项为零；
2. n 变量的全体最小项之和为 1；
3. 不同的最小项相与，结果为 0；
4. 相邻最小项相“或”可以合并成一项，并消去一个变量因子。

定义 2.9 (最小项相邻)

两最小项如仅有一个变量因子不同，其他变量均相同，则称这两个最小项相邻。例如 $\overline{A}BC+ABC=BC$。任一 n 变量的最小项，必定和其他 n 个不同最小项相邻。

2.6.3 最大项

定义 2.10 (最大项)

最大项是“或”项，具有以下特点：

1. 每个最大项一定包含 n 个因子的“或”项；
2. 每个变量必须以原变量或反变量形式出现**一次且仅一次**。

例如：两变量有 $A+B$、$A+\overline{B}$、$\overline{A}+B$、$\overline{AB}$ 四个最大项。

定义 2.11 (最大项的编号)

任一最大项用 M_i 表示，下标 i 为使该最大项为 **0** 的变量取值所对应的等效十进制数。

例如：使最大项 $\overline{A}+B+C$ 为 0 的取值为 $A,B,C=1,0,0$，二进制数 100 等效十进制数为 4，所以 $\overline{A}+B+C=M_4$。

性质 2.4 (最大项的性质)

1. 变量任取一组值，仅有一个最大项为 0，其他最大项为 1；
2. n 变量的全体最大项之积为 0；
3. 不同的最大项相或，结果为 1；
4. 两相邻的最大项相“与”可以合并成一项，并消去一个变量因子。例如 $\left(A+B+\overline{C}\right)(A+B+C)=A+B$。任一 n 变量的最大项，必定和其他 n 个不同最大项相邻。

2.6.4 最小项和最大项的关系

定理 2.5 (最小项与最大项的关系)

编号下标相同的最小项和最大项互为反函数，即

$$M_i=\overline{m_i} \tag{2.16}$$

2.6.5 逻辑函数的最小项之和形式

定理 2.6 (最小项之和形式)

最小项之和式为“与或”式，任一逻辑函数都可以表达为最小项之和的形式，而且是**唯一**的。例如

$$F(A,B,C)=\overline{A}B\overline{C}+A\overline{B}\overline{C}+AB\overline{C}=\Sigma m(2,4,6) \tag{2.17}$$

例子 2.1

将 $F(A,B,C)=AB+\overline{A}C$ 化为最小项之和的形式：

$$\begin{aligned}F(A,B,C)&=AB+\overline{A}C\\&=AB\left(C+\overline{C}\right)+\overline{A}C\left(B+\overline{B}\right)\\&=ABC+AB\overline{C}+\overline{A}BC+\overline{A}\overline{B}C\\&=\Sigma m(1,3,6,7)\end{aligned} \tag{2.18}$$

2.6.6 逻辑函数的最大项之积形式

定理 2.7 (最大项之积形式)

最大项之积式为“或与”式，任一逻辑函数都可以表达为最大项之积的形式，而且是**唯一**的。例如

$$F(A,B,C)=(A+B+C)\left(A+\overline{B}+C\right)\left(\overline{A}+B+C\right)=\Pi M(0,2,4) \tag{2.19}$$

例子 2.2

将 $F(A,B,C)=\left(\overline{A}+C\right)\left(B+\overline{C}\right)$ 化为最大项之积的形式：

$$\begin{aligned}F(A,B,C)&=\left(\overline{A}+C\right)\left(B+\overline{C}\right)\\&=\left(\overline{A}+C+B\overline{B}\right)\left(B+\overline{C}+A\overline{A}\right)\\&=\left(\overline{A}+B+C\right)\left(\overline{A}+\overline{B}+C\right)\left(A+B+\overline{C}\right)\left(\overline{A}+B+\overline{C}\right)\\&=\Pi M(1,4,5,6)\end{aligned} \tag{2.20}$$

2.6.7 两种标准形式之间的关系

定理 2.8 (两种标准形式之间的关系)

若 $F=\Sigma m_i$，则

$$\overline{F}=\sum_{j\neq i}m_j=\prod_{j\neq i}M_j \tag{2.21}$$

例子 2.3

$$F(A,B,C)=\Sigma(1,3,4,6,7)=\Pi(0,2,5) \tag{2.22}$$

2.7 真值表与逻辑表达式

真值表与逻辑表达式都是表示逻辑函数的方法。

2.7.1 由逻辑函数式列真值表

例子 2.4

试列出逻辑函数式 $F(A,B,C)=AB+BC$ 的真值表。

方法一（直接代入法）：把 A,B,C 三个变量所有取值的组合分别代入函数式，算出函数值，填进真值表，适用于变量较少的情况。

方法二：先将函数式 F 表示为最小项之和的形式：

$$\begin{aligned}F(A,B,C)&=AB+BC\\&=AB\left(C+\overline{C}\right)+BC\left(A+\overline{A}\right)\\&=ABC+AB\overline{C}+\overline{A}BC\\&=\Sigma m(3,6,7)\end{aligned} \tag{2.23}$$

最后根据最小项的性质，在真值表中对应于 ABC 取值为 011、110、111 处填“1”，其它位置填“0”，即

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

表 2.11 $F(A,B,C)=AB+BC$ 的真值表

方法三：根据函数式 F 的含义，直接填表。

2.7.2 由真值表写逻辑函数式

> **方法**
>
> 根据最小项的性质，用观察法，可直接从真值表写出函数的最小项之和表达式。

2.8 逻辑函数的化简

> **最简与或表达式的标准**
>
> 1. 所得与或表达式中，乘积项（与项）数目最少；
> 2. 每个乘积项中所含的变量数最少。

2.8.1 公式法

公式法化简就是反复运用逻辑代数公式消去多余的乘积项和每个乘积项中多余的因子，使函数式符合最简标准。常用方法见表 2.12。

方法	利用公式	例
并项法	$AB+A\overline{B}=A$	$F=\overline{A}\overline{B}C+\overline{A}BC+A\overline{B}C+ABC$ $=\left(\overline{A}\overline{B}+\overline{A}B\right)C+\left(A\overline{B}+AB\right)C$ $=\overline{A}C+AC=C$
吸收法	$A+AB=A$	$F=A\overline{B}+A\overline{B}C=A\overline{B}$
消项法	$AB+\overline{A}C+BC=AB+\overline{A}C$	$F=AB+\overline{A}C+BC=AB+\overline{A}C$
消因子法	$A+\overline{A}B=A+B$	$F=A+\overline{A}BC=A+BC$
配项法	$A+\overline{A}=1$、$A\cdot 1=A$	$F=AB+\overline{A}C+BC$ $=AB+\overline{A}C+\left(A+\overline{A}\right)BC$ $=AB(1+C)+\overline{A}C(1+B)=AB+\overline{A}C$

表 2.12　公式法化简的常用方法

> **注意**
>
> 对比较复杂的函数式，要求熟练掌握上述方法，才能把函数化成最简。

2.8.2 卡诺图法

将逻辑函数用**卡诺图**表示，然后在卡诺图上进行化简。卡诺图是一种包含小方块的几何图形，每个小方块称为一个**单元**，每个单元对应一个最小项，两个相邻的最小项在卡诺图中也必须相邻。

2.8.2.1 卡诺图的构成

- 相邻的含义：① **几何相邻性**（左右紧挨或者上下相接）；② **对称相邻性**（图形中对称位置的单元是相邻的）；
- 行列标号按**循环码**（格雷码）排列。

三变量卡诺图与四变量卡诺图分别见图 2.11 和图 2.12。

$A\setminus BC$	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

图 2.11　三变量卡诺图

$AB \setminus CD$	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

图 2.12 四变量卡诺图

二变量、五变量卡诺图类似，标号同样按循环码排列。

2.8.2.2 逻辑函数的卡诺图表示法

把各组变量值所对应的逻辑函数 F 的值填在对应的小方格中（卡诺图实质是真值表的另一种画法）。

2.8.2.3 在卡诺图上合并最小项的规则

- 任何**两个**标 1 方格相邻，可合并为 1 项，消去 1 个变量；
- 任何**四个**标 1 方格相邻，可合并为一项，消去 2 个变量（四个标 1 方格相邻的特点：① 同在一行或一列；② 同在一个田字格中）；
- 任何**八个**标 1 方格相邻，可并成一项，消去 3 个变量。

一般规律：在 n 个变量的卡诺图中，只有 2^i 个相邻的标 1 方格（必须排列成方形格或矩形格）才能圈在一起合并为一项，该项保留原来各项中 $n-i$ 个相同的变量，消去 i 个不同的变量。

2.8.2.4 用卡诺图化简逻辑函数

最简标准：项数最少（圈数最少）；每项中变量数最少（圈尽可能大）。

例子 2.5

将 $F(A,B,C)=\Sigma m(3,4,5,6,7)$ 化为最简与或式。

$A \setminus BC$	00	01	11	10
0	0	0	1	0
1	1	1	1	1

图 2.13 例 1 的卡诺图及圈法

解. 将 $A=1$ 的 4 个标 1 方格圈为一组（图中蓝色圈），合并为乘积项 A；将 $BC=11$ 的 m_3、m_7 两个标 1 方格圈为一组（图中红色圈），合并为乘积项 BC。两圈已覆盖全部标 1 方格，故最简与或式为

$$F(A,B,C)=A+BC \tag{2.24}$$

例子 2.6

将 $F(A,B,C,D)=\Sigma m(0,1,3,7,8,10,13)$ 化为最简与或式。

$AB \setminus CD$	00	01	11	10
00	1	1	1	0
01	0	0	1	0
11	0	1	0	0
10	1	0	0	1

图 2.14 例 2 的卡诺图及圈法

解. 先圈出孤立的标 1 方格，再圈出覆盖其余标 1 方格的最大圈：

1. m_0, m_1 相邻，圈为一组（蓝色圈），合并为 $\overline{A}\overline{B}\overline{C}$；
2. m_3, m_7 相邻，圈为一组（红色圈），合并为 $\overline{A}CD$；
3. m_8, m_{10} 位于同一行两端、左右相邻（卷绕相邻），圈为一组（橙色圈），合并为 $A\overline{B}\overline{D}$；
4. m_{13} 孤立，单独圈出（绿色圈），为 $AB\overline{C}D$。

将各乘积项相加，得最简与或式

$$F(A,B,C,D)=\overline{A}\overline{B}\overline{C}+\overline{A}CD+A\overline{B}\overline{D}+AB\overline{C}D \tag{2.25}$$

2.8.2.5 化简中注意的问题

1. 每一个标 1 的方格必须至少被圈一次；
2. 每个圈中包含的相邻小方格数必须为 2 的整数次幂；
3. 为了得到尽可能大的圈，圈与圈之间可以重叠；
4. 若某个圈中的所有标 1 方格已经完全被其他圈所覆盖，则该圈为多余的。

2.8.2.6 用卡诺图求反函数的最简与或式

在卡诺图中合并标 **0** 方格，可得到反函数的最简与或式；常利用该方法求逻辑函数 F 的最简与或非式（将 $\overline{F}$ 上的非号移到右边）。

2.8.2.7 复杂逻辑函数

可将函数分解成多个部分，先将每个部分分别填入各自的卡诺图，然后通过卡诺图对应方格的运算（与、或、异或等）求出函数的卡诺图，再进行化简。

2.8.2.8 不完全确定的逻辑函数及其化简（无关项）

定义 2.12 (无关项)

在某些实际数字电路中，逻辑函数的输出只和一部分最小项有确定对应关系，而和余下的最小项无关；余下的最小项无论写入函数式与否都不影响电路的逻辑功能。这些最小项称为**无关项**，用字母 d（don't care）表示，对应的函数值记为“×”。

包含无关项的逻辑函数称为**不完全确定的逻辑函数**。利用无关项往往可以把函数化得更简单。

例子 2.7

设计一个奇偶判别电路，输入为 8421BCD 码，输入为偶数时输出 0，输入为奇数时输出 1。由于 8421BCD 码中无 1010 ~ 1111 这 6 个码，电路禁止输入这 6 个码，它们对应的最小项为无关项：

$$F(A,B,C,D)=\Sigma m(1,3,5,7,9)+\Sigma d(10\sim 15) \tag{2.26}$$

- 若不利用无关项（将卡诺图中的 × 均作 0 处理），化简结果为

$$F(A,B,C,D)=\overline{A}D+\overline{BC}D \tag{2.27}$$

- 若利用无关项（将 × 按化简需要处理，有的作 0、有的作 1），化简结果为 $F(A,B,C,D)=D$。

第三章 组合逻辑电路基础

定义 3.1 (组合逻辑电路)

电路在任意时刻的输出状态仅由当前输入状态决定，与之前的输出状态和输入状态无关。

3.1 由基本逻辑门构成的组合电路的设计与分析

3.1.1 一般分析方法

分析步骤：

1. 根据逻辑电路，写出输出逻辑函数的表达式；
2. 列出真值表；
3. 由真值表或表达式分析电路功能。

例子 3.1

分析下图所示逻辑电路。

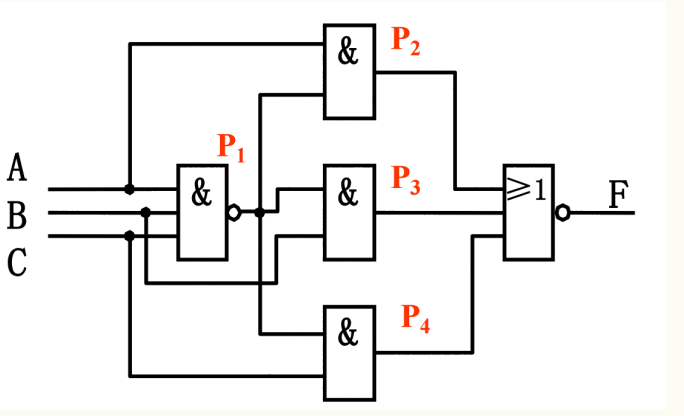


图 3.1　待分析的组合逻辑电路

采取从后往前分析的方法，把各逻辑门的输出分别标为 P_1、P_2、P_3、P_4，逐个写出表达式：

$$F=\overline{P_2+P_3+P_4},\quad P_2=AP_1,\quad P_3=BP_1,\quad P_4=CP_1,$$

$$P_1=\overline{ABC}$$

代入并化简：

$$\begin{aligned}F&=\overline{(A+B+C)\cdot\overline{ABC}}\\&=\overline{A+B+C}+ABC\\&=\overline{ABC}+ABC\end{aligned}$$

由表达式列出真值表，其中 $P_1\sim P_4$ 为图中各逻辑门的输出：

A	B	C	P_1	P_2	P_3	P_4	F
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	1	0	1	0	1	0	0
0	1	1	1	0	1	1	0
1	0	0	1	1	0	0	0
1	0	1	1	1	0	1	0
1	1	0	1	1	1	0	0
1	1	1	0	0	0	0	1

表 3.1　组合逻辑电路例题的真值表

解. 由真值表可见，当且仅当 $A=B=C$ 时 $F=1$，否则 $F=0$。即三个输入取值一致时输出为 1，不一致时输出为 0，该电路实现三变量的“一致”判断（同或）功能。

3.1.2 一般设计方法

一般步骤：

1. 由实际问题列出真值表；
2. 由真值表写出逻辑表达式；
3. 化简、变换输出逻辑表达式；
4. 画出逻辑图。

如果现在要设计用与非门的一个三变量表决电路，表决规则为少数服从多数，怎么设计？

设由 A、B、C 表示三个输入变量，F 表示表决结果。并设 A、B、C 为 1 表示赞成，为 0 表示反对；F 为 1 表示表决通过，为 0 表示不通过。列出真值表：

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

表 3.2　三变量表决器真值表

由表决规则，三个输入中有两个或两个以上为 1 时表决通过，故 $F=1$ 对应 ABC 取 011、101、110、111 四种情况。

将 $F=1$ 的四个最小项 m_3、m_5、m_6、m_7 填入卡诺图并圈组，如图所示：

$A \setminus BC$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

图 3.2　三变量表决电路的卡诺图及圈法

图中三个圈分别合并为 BC、AC、AB，故最简与或式为

$$F = AB + BC + AC = \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}}$$

下一步画出逻辑图，见图 3.3。

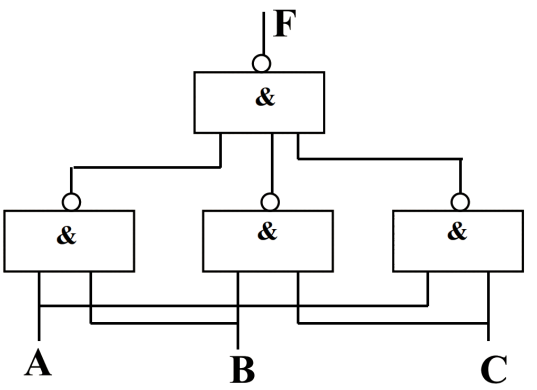


图 3.3　三变量表决电路的逻辑图

3.1.3 自顶向下的模块化设计方法

顶：指系统功能，即系统总要求，较抽象。

向下：指根据系统总要求，将系统分解为若干个子系统，再将每个子系统分解为若干个功能模块，直至分成许多各具特定功能的基本模块为止。

如果我现在想设计一个数据检测系统，功能表如下：

S_1	S_2	输出功能
0	0	$A+B$
0	1	$A-B$
1	0	$\mathrm{Min}(A,B)$
1	1	$\mathrm{Max}(A,B)$

表 3.3　数据检测系统的功能表

其中 S_1、S_2 为两个子系统，A、B 为传感器数据输入。

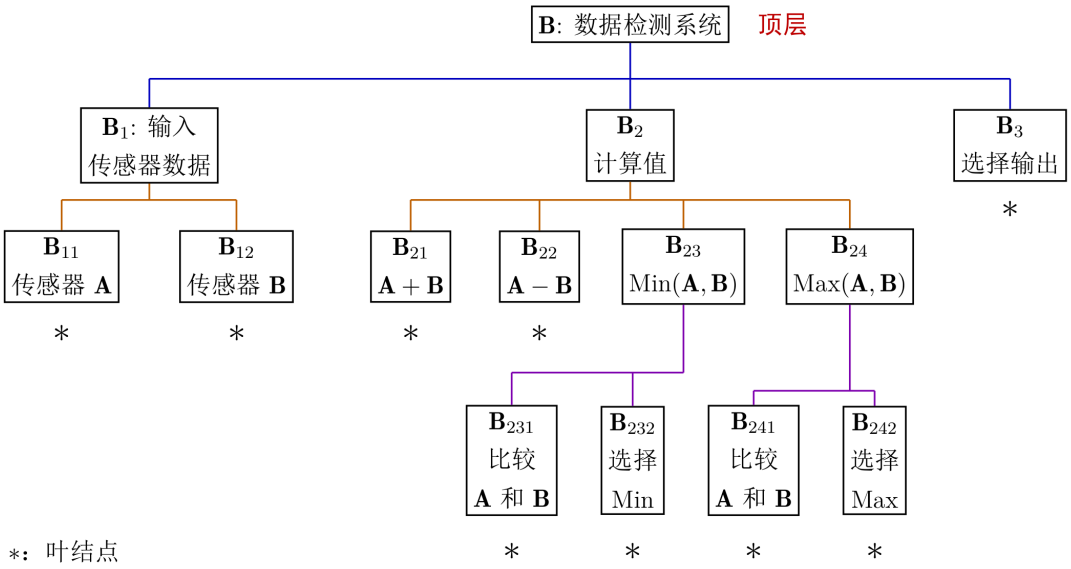


图 3.4　数据检测系统的分层设计树

各模块之间的信号连接关系见图 3.5。

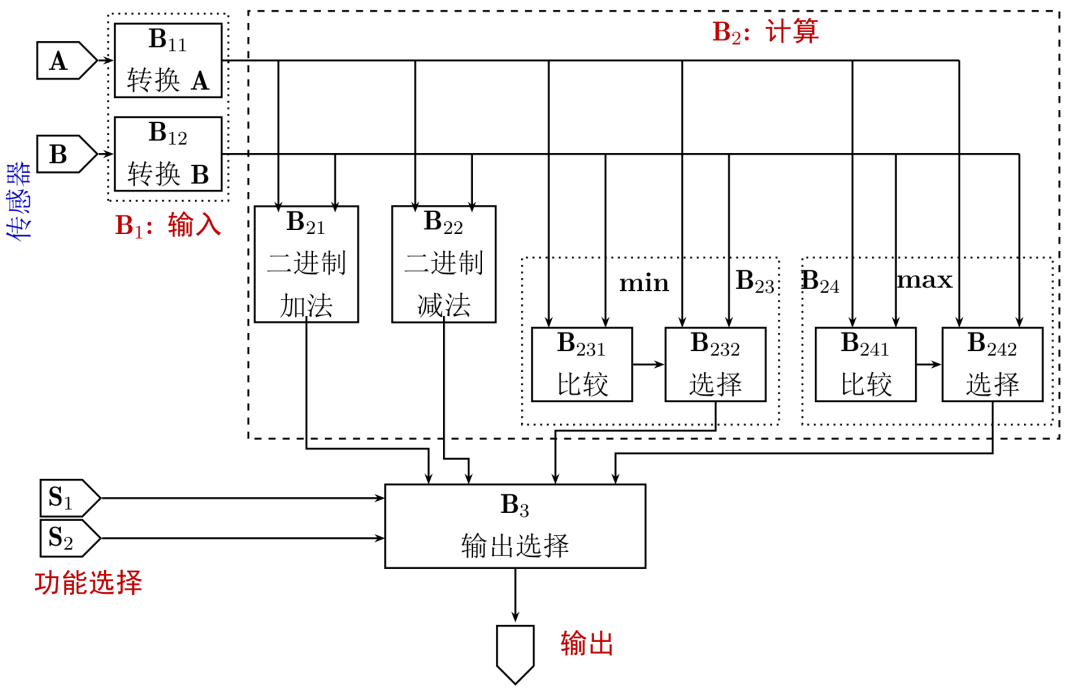


图 3.5　数据检测系统的分层方框图

这样，我们就完成了一个系统的模块化设计。

3.2 MSI 构成的组合逻辑电路

定义 3.2 (MSI)

中规模集成电路。

使用 MSI 进行模块化可以有效避免重复造轮子，使用封装好的集成电路进行组合，可以节省资源和时间。

3.2.1 编码器

数字设备一般只能处理二进制码信息，因此对需要处理的任何信息，都必须要转换成符合一定规则的二进制码。

而我们的编码器能够将信息（如数和字符等）转换成符合一定规则的二进制代码。

3.2.1.1 二进制编码器

用 n 位二进制代码对 $N = 2^n$ 个特定信息进行编码的逻辑电路。

下面说明一下二进制编码器的设计方法。

假设咱们现在需要设计一个具有互相排斥输入条件的编码器。

输入为 X_0、X_1、X_2、X_3，输出为 A_0、A_1。具有如下对应关系：

输入	A_1	A_0
X_0	0	0
X_1	0	1
X_2	1	0
X_3	1	1

表 3.4 二进制编码器的输入输出对应关系

由于编码器输入互相排斥，任一时刻只有 0001、0010、0100、1000 四种输入组合会出现，其余 12 种组合均不会出现，其对应输出可任意取值，称为**约束项**（任意项），用 × 表示。由此列出真值表：

X_3	X_2	X_1	X_0	A_1	A_0
0	0	0	0	×	×
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	×	×
0	1	0	0	1	0
0	1	0	1	×	×
0	1	1	0	×	×
0	1	1	1	×	×
1	0	0	0	1	1
1	0	0	1	×	×
1	0	1	0	×	×
1	0	1	1	×	×
1	1	0	0	×	×
1	1	0	1	×	×
1	1	1	0	×	×
1	1	1	1	×	×

表 3.5 二进制编码器的真值表

将约束项按任意取值处理，分别填入 A_1、A_0 的卡诺图，并把标 1 方格与约束项一起圈成尽可能大的组：

$X_3X_2 \setminus X_1X_0$	00	01	11	10
00	×	0	×	0
01	1	×	×	×
11	×	×	×	×
10	1	×	×	×

$$A_1 = X_2 + X_3$$

$X_3X_2 \setminus X_1X_0$	00	01	11	10
00	×	0	×	1
01	0	×	×	×
11	×	×	×	×
10	1	×	×	×

$$A_0 = X_1 + X_3$$

图 3.6　二进制编码器输出函数的卡诺图及圈法（上：A_1，下：A_0）

由图 3.6 可见：A_1 的卡诺图中，$X_2 = 1$ 的两行（X_3X_2 取 01、11）圈组得 X_2，$X_3 = 1$ 的两行（X_3X_2 取 11、10）圈组得 X_3，两组合用 11 行；A_0 的卡诺图中，$X_1 = 1$ 的两列（X_1X_0 取 11、10）圈组得 X_1，$X_3 = 1$ 的两行圈组得 X_3。故得最简与或式

$$A_1 = X_2 + X_3,\quad A_0 = X_1 + X_3$$

这是一个 **4 线——2 线**编码器。电路图如下：

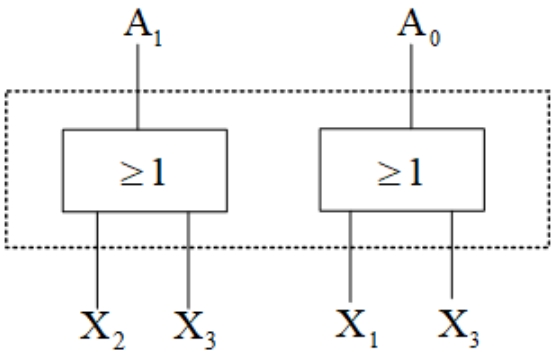


图 3.7　二进制编码器示例

带输出使能（Enable）端的优先编码器：

- 输出使能端：用于判别电路是否有信号输入；
- 优先：对输入信号按轻重缓急排序，当有多个信号同时输入时，只对优先权高的一个信号进行编码。

把上面的 4 线——2 线编码器改成带输出使能端的优先编码器，约定优先级为：X_3, X_2, X_1, X_0。

X_3	X_2	X_1	X_0	A_1	A_0	E_0
0	0	0	0	0	0	1
0	0	0	1	0	0	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	1	0	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	1	1	0
1	0	0	1	1	1	0
1	0	1	0	1	1	0
1	0	1	1	1	1	0
1	1	0	0	1	1	0
1	1	0	1	1	1	0
1	1	1	0	1	1	0
1	1	1	1	1	1	0

表 3.6 带使能端二进制编码器的真值表

$X_3X_2 \setminus X_1X_0$	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$$A_1 = X_2 + X_3$$

$X_3X_2 \setminus X_1X_0$	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

$$A_0 = X_1 + X_3$$

图 3.8 带使能的二进制编码器输出函数的卡诺图及圈法（上：A_1，下：A_0）

由图 3.8 可见，带使能端后输出函数的最简与或式仍为

$$A_1 = X_2 + X_3, \quad A_0 = X_1 + X_3$$

3.2.1.2 二—十进制编码器

输入： $I_0, I_1, I_2, \cdots, I_9$，表示十个要求编码的信号。

输出： BCD 码。

3.2.1.3 通用编码器集成电路

两种主要集成电路：

1. 10 线—4 线优先编码器，如 74147（TTL 电路）；
2. 8 线—3 线优先编码器，如 74148（TTL 电路）。

这两种优先编码器逻辑符号如图 3.9 所示

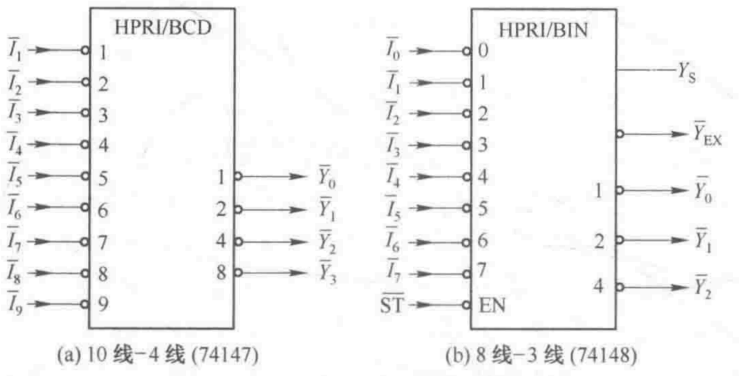


图 3.9 两种优先编码器的逻辑符号

- 对于 **74147**,注意观察，由于 $\overline{I}_0$的编码，同其他各输入端均为 1 时是等效的，所以在集成电路中省去了它。
- 对于 **74148**：8 线−3 线优先编码器，输入、输出均为低电平有效。除 $\overline{I}_7, \overline{I}_6, \cdots, \overline{I}_0$ 八路输入端和 $\overline{A}_2, \overline{A}_1, \overline{A}_0$ 三路输出端外，还增设了三个控制端：
  1. **使能输入** $\overline{ST}$：只要它为 1，芯片就不工作，输出及 $\overline{Y}_{EX}$、Y_S 全为 1；
  2. **选通输出** Y_S：$\overline{ST}=0$ 且至少有一路输入有效时，$Y_S=0$，表示本级有编码输出；
  3. **扩展输出** $\overline{Y}_{EX}$：八路输入全为 1（无有效输入）时，$Y_S=1$ 而 $\overline{Y}_{EX}=0$，专供级联扩展使用。

  八路输入的优先级由 $\overline{I}_7$ 到 $\overline{I}_0$ 依次降低。芯片正常工作时，输出为有效输入序号的反码，例如 $\overline{I}_5=0$ 时，$5=(101)_2$，输出 $\overline{A}_2\overline{A}_1\overline{A}_0=010$。

  正因为 Y_S 与 $\overline{Y}_{EX}$ 能把“$\overline{I}_0$ 有效”和“八路均无输入”这两种都输出 111 的情形区分开，74148 保留了 $\overline{I}_0$，不像 74147 那样把它省去。

$\overline{ST}$	有效输入	$\overline{A}_2$	$\overline{A}_1$	$\overline{A}_0$	Y_S	$\overline{Y}_{EX}$
1	×	1	1	1	1	1
0	$\overline{I}_7$	0	0	0	0	1
0	$\overline{I}_6$	0	0	1	0	1
0	$\overline{I}_5$	0	1	0	0	1
0	$\overline{I}_4$	0	1	1	0	1
0	$\overline{I}_3$	1	0	0	0	1
0	$\overline{I}_2$	1	0	1	0	1
0	$\overline{I}_1$	1	1	0	0	1
0	$\overline{I}_0$	1	1	1	0	1
0	无（八路全为 1）	1	1	1	1	0

表 3.7 74148 的功能表

表中“有效输入”一栏指当前优先级最高的那一路低电平输入端，其余输入端取任意值。

设置 $\overline{Y}_{EX}$、$\overline{ST}$ 的目的就是便于级联扩展：把两片 74148 并成 16 路输入，高位片（接 $\overline{I}_{15},\cdots,\overline{I}_8$）的 $\overline{Y}_{EX}$ 接低位片的 $\overline{ST}$，高位片的 Y_S 作为第四位输出 $\overline{A}_3$，就得到 16 线–4 线优先编码器。这样高位片一旦有输入，$\overline{Y}_{EX}=0$ 便封住低位片，整片的优先级顺序不会被破坏。

3.2.2 译码器

定义 3.3 (译码器)

译码是编码的逆过程，其作用是将一组代码转换为确定的信息。二进制译码器的输入为 n 位二进制代码，输出为 2^n 个特定信息，常称为 n 线–2^n 线译码器。

3.2.2.1 二进制译码器

以 2 线–4 线译码器为例。输入为 B（高位）、A（低位），输出为 $Y_0\sim Y_3$。高电平输出有效时，各输出分别为一个最小项：

$$Y_0=\overline{B}\,\overline{A}=m_0,\quad Y_1=\overline{B}A=m_1,\quad Y_2=B\overline{A}=m_2,\quad Y_3=BA=m_3$$

高电平输出有效的真值表如下：

B	A	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

表 3.8 高电平输出有效的 2 线–4 线译码器真值表

若输出低电平有效，则输出端取反：

$$\overline{Y_i} = \overline{m_i}, \quad i = 0, 1, 2, 3$$

结论

1. 高电平输出有效的二进制译码器：$Y_i = m_i$；
2. 低电平输出有效的二进制译码器：$\overline{Y_i} = \overline{m_i}$，即每个输出端都是一个最小项的非。

3.2.2.1.1 用译码器实现组合逻辑函数

性质 3.1 (基本原理)

二进制译码器能产生输入信号的全部最小项，而任一组合逻辑函数都可以写成最小项之和的形式，因此用译码器加适当的门电路即可实现组合逻辑函数。

例如，试用 3 线–8 线译码器和逻辑门实现

$$F(Q, X, P) = \Sigma m(0, 1, 4, 6, 7) = \Pi M(2, 3, 5)$$

把 Q、X、P 依次接到译码器的高位到低位地址端（$Q \to A_2$、$X \to A_1$、$P \to A_0$）。实现方法有以下四种：

译码器类型	配合的门	实现方法
高电平有效	或门	$F = m_0 + m_1 + m_4 + m_6 + m_7$，把对应输出直接送或门
低电平有效	与非门	$F = \overline{\overline{m_0}\,\overline{m_1}\,\overline{m_4}\,\overline{m_6}\,\overline{m_7}}$，把对应输出送与非门
高电平有效	或非门	$F = \overline{m_2 + m_3 + m_5}$，把函数值为 0 的最小项送或非门
低电平有效	与门	$F = \overline{m_2}\,\overline{m_3}\,\overline{m_5}$，把函数值为 0 的最小项送与门

表 3.9 用 3 线–8 线译码器实现 $F(Q, X, P) = \Sigma m(0, 1, 4, 6, 7)$ 的四种方案

由表 3.9 可见，用译码器实现逻辑函数时，既可以直接用“1”对应的最小项，也可以利用“0”对应的最小项，具体取哪一种取决于译码器输出有效电平和所选门电路的类型。

3.2.2.1.2 译码器的使能控制输入端

在译码器上加使能控制端 EN，可以控制电路是否工作，也便于扩展译码器的容量。高电平输出有效的译码器满足

$$Y_i = m_i \cdot EN$$

$EN = 1$ 时正常译码；$EN = 0$ 时所有输出均为 0。低电平输出有效的译码器在禁止时所有输出均为 1。

利用使能端可以方便地扩展容量。例如用两片 2 线−4 线译码器组成 3 线−8 线译码器：把输入 I_2 接第一片的 EN、经反相后接第二片的 EN。当 $I_2 = 0$ 时第一片工作、第二片禁止；当 $I_2 = 1$ 时第一片禁止、第二片工作，两片的输出合起来正好是 8 路译码输出。

3.2.2.2 二-十进制译码器

二-十进制译码器又称 4 线−10 线译码器，输入为 8421BCD 码，输出为十个高、低电平信号。

输出低电平有效时

$$\overline{Y_i} = \overline{m_i}, \quad i = 0, 1, \cdots, 9$$

即输入为 i 时只有 $\overline{Y_i} = 0$，其余输出均为 1。对于 1010 ~ 1111 六个不会出现的伪码，译码器不产生有效输出，十个输出全为 1，从而拒绝伪码。

输入 $A_3A_2A_1A_0$	有效输出（低电平）
0000	Y_0
0001	Y_1
0010	Y_2
0011	Y_3
0100	Y_4
0101	Y_5
0110	Y_6
0111	Y_7
1000	Y_8
1001	Y_9
1010 ~ 1111	无（十个输出全为 1）

表 3.10 4 线−10 线译码器的输出对应关系

3.2.2.3 通用译码器集成电路

常用的集成译码器有 74138、74154、7442 等。

74138：带使能端的 3 线−8 线译码器，输出低电平有效，S_1 高电平有效，S_2、S_3 低电平有效。当 $S_1=1$ 且 $S_2=S_3=0$ 时电路正常译码；否则电路禁止工作，八个输出全为 1。

S_1	S_2	S_3	地址 $A_2A_1A_0$	输出
0	×	×	×	全为 1
×	1	×	×	全为 1
×	×	1	×	全为 1
1	0	0	000	$\overline{Y_0}=0$，其余为 1
1	0	0	001	$\overline{Y_1}=0$，其余为 1
1	0	0	010	$\overline{Y_2}=0$，其余为 1
1	0	0	011	$\overline{Y_3}=0$，其余为 1
1	0	0	100	$\overline{Y_4}=0$，其余为 1
1	0	0	101	$\overline{Y_5}=0$，其余为 1
1	0	0	110	$\overline{Y_6}=0$，其余为 1
1	0	0	111	$\overline{Y_7}=0$，其余为 1

表 3.11 74138 的功能表

利用使能端可以扩展位数。用两片 74138 构成 4 线−16 线译码器时，地址 $a_2a_1a_0$ 并联接到两片，最高位 a_3 接低位片（输出 $Y_0\sim Y_7$）的 S_2，经反相后接高位片（输出 $Y_8\sim Y_{15}$）的 S_2，两片均取 $S_1=1$、$S_3=0$。当 $a_3=0$ 时低位片工作，当 $a_3=1$ 时高位片工作。

74154：4 线−16 线译码器，输出低电平有效，带两个使能端。 **7442**：4 线−10 线译码器，输出低电平有效。

3.2.2.4 译码器做数据分配器

数据分配器的功能与数据选择器相反：把一个输入通道上的信号送到多个输出端中的某一个。

以 1 分 4 数据分配器为例，D 为数据输入，A_1A_0 为地址，EN 为使能端：

EN	A_1	A_0	Y_0	Y_1	Y_2	Y_3
0	×	×	0	0	0	0
1	0	0	D	0	0	0
1	0	1	0	D	0	0
1	1	0	0	0	D	0
1	1	1	0	0	0	D

表 3.12　1 分 4 数据分配器的功能表

其输出表达式为

$$Y_i = (m_i \cdot D) \cdot EN, \quad i = 0, 1, 2, 3$$

由此可见，数据分配器可以用带使能端的译码器实现：把数据 D 接到译码器的使能端，地址接译码器的地址端，译码器的各输出就是分配到各通道的数据。例如用 74154 实现 1 分 16 数据分配器，可令 $S_1 = D$、$S_0 = 0$，则 $Y_i = m_i D$。

3.2.2.5 显示译码器

显示译码器把表示数字的 BCD 码转换成七段显示码，用来驱动七段数码管。

七段数码管由 a、b、c、d、e、f、g 七个发光段组成，分为两类：

1. **共阴**：七个发光管的阴极连在一起接地，阳极加高电平时该段点亮；
2. **共阳**：七个发光管的阳极连在一起接高电平，阴极加低电平时该段点亮。

以输入 8421BCD 码、驱动共阳显示器为例，设计步骤如下：

1. 列真值表；
2. 化简、写最简函数表达式；
3. 画电路图。

D	C	B	A	a	b	c	d	e	f	g	显示
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0	2
0	0	1	1	0	0	0	0	1	1	0	3
0	1	0	0	1	0	0	1	1	0	0	4
0	1	0	1	0	1	0	0	1	0	0	5
0	1	1	0	0	1	0	0	0	0	0	6
0	1	1	1	0	0	0	1	1	1	1	7
1	0	0	0	0	0	0	0	0	0	0	8
1	0	0	1	0	0	0	0	1	0	0	9

表 3.13 8421BCD 码—七段显示码真值表（共阳，低电平点亮）

表中 $a \sim g$ 为低电平有效的段驱动信号。把它们取反，得到高电平有效的段信号，按高电平有效化简（1010 ~ 1111 作无关项，并兼顾多输出函数的公共项），得

$$a = A + C + BD + \overline{BD}$$

$$b = \overline{B} + CD + \overline{C}\,\overline{D}$$

$$c = B + D + \overline{C}$$

$$d = A + C\overline{B} + C\overline{D} + \overline{B}\,\overline{D} + BD\overline{C}$$

$$e = C\overline{D} + \overline{BD}$$

$$f = A + B\overline{C} + B\overline{D} + \overline{C}\,\overline{D}$$

$$g = A + B\overline{C} + \overline{B}C + C\overline{D}$$

共阳显示器直接使用表 3.13 中的低有效编码；若驱动共阴显示器，则把上式的结果直接送到各段即可。

> **说明**
>
> 化简时既利用了无关项，又考虑了多输出函数化简中的公共项，因此上式不一定是对每个输出单独化简所得的最简式，但可以使整个译码器的门电路最省。

3.2.3 数据选择器

定义 3.4 (数据选择器（MUX）)

数据选择器的功能是从多路输入数据中选择其中的一路送至输出端，简称 MUX。数据输入端的数目称为通道数，常见的有二选一、四选一、八选一、十六选一等。

3.2.3.1 数据选择器电路结构

一个 N 选一的数据选择器有 N 路数据输入端、一路数据输出端和 K 路地址码输入端，且 $2^K = N$。

以四选一数据选择器为例，$D_0 \sim D_3$ 为数据输入，A_1A_0 为地址输入，Y 为输出：

A_1	A_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

表 3.14 四选一数据选择器的功能表

由功能表可直接写出

$$Y = \overline{A_1A_0}D_0 + \overline{A_1}A_0D_1 + A_1\overline{A_0}D_2 + A_1A_0D_3 = \sum_{i=0}^{3} m_iD_i$$

其电路由地址译码器（产生四个最小项）和四个与门、一个或门组成：每个最小项与对应的数据相乘，再把各乘积项相加。

利用地址端的译码作用可以扩展通道数。例如用五片四选一数据选择器组成十六选一数据选择器：低两位地址 A_1A_0 并联接到五片的地址端，高两位 A_3A_2 经 2 线—4 线译码器分别选通前四片，第五片作为扩展级，把前四片的输出再选通到总输出。

3.2.3.2 通用数据选择器集成电路

常用 MUX 集成电路见表 3.15。

输入路数	TTL 型号	特点
16	74150	十六选一
8	74151	八选一，带反码输出
4	74153	双四选一，两单元共用地址
2	74157	四组二选一

表 3.15 常用数据选择器集成电路

双四选一 MUX 74153：内部有两个相同的四选一单元，共用地址端 A_1A_0，每个单元有独立的低电平有效选通端 $\overline{ST}$。每个单元的输出为

$$Y = \left(\overline{A_1A_0}D_0 + \overline{A_1}A_0D_1 + A_1\overline{A_0}D_2 + A_1A_0D_3\right)\cdot\overline{ST}$$

即 $Y = (\sum m_iD_i)\cdot\overline{ST}$。

八选一 MUX 74151：有 $D_0 \sim D_7$ 八路数据输入、$A_2A_1A_0$ 三路地址输入、低电平有效选通端 $\overline{ST}$，并同时给出反码输出 $\overline{Y}$。其输出为

$$Y = \left(\sum_{i=0}^{7} m_iD_i\right)\cdot\overline{ST}$$

3.2.3.3 数据选择器的应用

3.2.3.3.1 用数据选择器实现组合逻辑函数

性质 3.2 (基本思想)

数据选择器的输出为 $Y = \sum m_iD_i$，地址变量产生全部最小项，通过给各数据输入端 D_i 赋不同的值（0、1 或某个变量），就可以选取组成逻辑函数所需的最小项。

设待实现函数的输入变量数为 L，所选 MUX 的地址端数为 K。

（1）$L = K$ 时：把 L 个变量直接接到地址端，数据端按函数的最小项情况接 0 或 1。

例子 3.2 (用 74151 实现 $F(A, B, C) = Sigma m(1, 2, 4, 7)$)

74151 的输出为

$$Y = \left(\overline{A_2A_1A_0}D_0 + \cdots + A_2A_1A_0D_7\right)\cdot\overline{ST}$$

令 $ST = 0$（芯片选通），$A_2 = A$、$A_1 = B$、$A_0 = C$，比较两式可得

$$D_0 = D_3 = D_5 = D_6 = 0,\quad D_1 = D_2 = D_4 = D_7 = 1$$

解. 按上述取值连接，MUX 的输出即为 $F(A,B,C)$。注意 MUX 必须被选通（$ST=0$），且变量与地址端的连接顺序必须与最小项编号一致。

（2）$L>K$ 时：从 L 个变量中任取 K 个作为地址变量，剩余变量通过数据端引入。

例子 3.3 (用四选一 MUX 实现三变量函数)

试用四选一 MUX 实现

$$F=\overline{AB}+A\overline{B}C+AB\overline{C}$$

四选一 MUX 选通时的输出为

$$Y=\overline{A_1A_0}D_0+\overline{A_1}A_0D_1+A_1\overline{A_0}D_2+A_1A_0D_3$$

以 A、B 作为地址变量（$A_1=A$、$A_0=B$），把 F 按 AB 的四种取值整理：

$$F=\overline{AB}\cdot 1+\overline{A}B\cdot 0+A\overline{B}\cdot C+AB\cdot\overline{C}$$

比较系数得

$$D_0=1,\quad D_1=0,\quad D_2=C,\quad D_3=\overline{C}$$

解. 该题的解法不唯一：也可以选 A、C 或 B、C 作地址变量，此时数据端的表达式随之改变。

例子 3.4 (用四选一 MUX 实现四变量函数)

试用四选一数据选择器实现

$$F(A,B,C,D)=\Sigma m(1,2,4,9,10,11,12,14,15)$$

取 $A_1=A$、$A_0=B$，按 AB 的四种取值把卡诺图分成四组：

$AB\setminus CD$	00	01	11	10
00	0	1	0	1
01	1	0	0	0
11	1	0	1	1
10	0	1	1	1

图 3.10 四变量函数的卡诺图（按 AB 分组）

于是

$$D_0 = C\overline{D} + \overline{C}D = C \oplus D$$

$$D_1 = \overline{C\,D}$$

$$D_2 = C + D$$

$$D_3 = C + \overline{D}$$

解. 把 $D_0 \sim D_3$ 按上式接到数据端，地址端接 A、B，即可实现该函数。地址变量选取不同，数据端的表达式也不同。

（3）$L < K$ 时：把多余的高位地址端接地，数据输入端按地址分组并联，使 MUX 的等效通道数降为 $N = 2^L$。

3.2.3.3.2 数据选择器的其他应用

1. **分时多路传输**：周期地改变地址输入，可以把多路信息分时地送到同一输出端。例如四选一 MUX 的四个数据端分别接 A、B、C、D，地址按 $00 \to 01 \to 10 \to 11$ 循环变化，输出端就依次得到 A、B、C、D 的时分复用信号。
2. **并行数码比较器**：用 74138 产生 B 的全部最小项，接到 74151 的数据端，地址端接 A。当 $A = B$ 时，被选中的那一路数据为 0，输出 $Y = 0$；否则 $Y = 1$。因此 $Y = 0$ 表示两数相等，$Y = 1$ 表示两数不等。

3.2.4 加法器

数字信号的算术运算主要有加、减、乘、除，其中加法是最基础的运算，因此算术运算电路的核心是加法器。

3.2.4.1 基本加法器电路

3.2.4.1.1 半加器

仅考虑两个一位二进制数相加，而不考虑低位进位，称为半加。设 A、B 为两个加数，S 为本位和，C 为本位向高位的进位，则

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

表 3.16 半加器的真值表

$$S = \overline{A}B + A\overline{B} = A \oplus B, \quad C = AB$$

3.2.4.1.2 全加器

多位数相加时，除本位的两个加数外，还要考虑低位向本位的进位。设 A_i、B_i 为本位加数，C_{i-1} 为低位向本位的进位，S_i 为本位和，C_i 为本位向高位的进位，则

A_i	B_i	C_{i-1}	C_i	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

表 3.17 一位全加器的真值表

由真值表得

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = A_iB_i + (A_i \oplus B_i)C_{i-1}$$

令半加和 $S = A_i \oplus B_i$，则 $S_i = S \oplus C_{i-1}$，$C_i = SC_{i-1} + A_iB_i$。因此一个全加器可以用两个半加器和一个或门实现。

3.2.4.2 串行进位加法器

把 n 个全加器串联起来，低位全加器的进位输出接到高位全加器的进位输入，就构成 n 位串行进位加法器（行波进位加法器）。其结构简单，但高位的运算必须等待低位的进位逐级传递，运算速度慢。

3.2.4.3 高速加法器

3.2.4.3.1 全并行加法器

按组合逻辑电路的一般设计方法，和 S 与最高位进位 C_O 总能直接写成输入信号 A、B 的最简函数表达式，因此可以用两级门电路实现。其速度最快，但电路复杂，位数稍多就无法采用。

3.2.4.3.2 超前进位加法器

超前进位加法器的设计思想是：通过逻辑电路提前得出加到每一位全加器上的进位输入信号，而无需从最低位开始逐位传递进位。

把全加器的进位表达式改写为

$$C_i = A_iB_i + (A_i + B_i)C_{i-1}$$

令

$$G_i = A_iB_i \quad (\text{进位产生项}), \quad P_i = A_i + B_i \quad (\text{进位传送项})$$

则

$$C_i = G_i + P_iC_{i-1}$$

对四位加法器逐级展开：

$$C_0 = G_0$$

$$C_1 = G_1 + P_1G_0$$

$$C_2 = G_2 + P_2G_1 + P_2P_1G_0$$

$$C_3 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

由 P_i、G_i 经过两级门电路就可求得全部进位信号。实际实现时，把求 G_i、P_i 的电路放进全加器中，而把全加器中逐级求进位的电路去掉；根据 G_i、P_i 求进位信号的电路称为超前进位电路（CLA）。

3.2.4.3.3 集成加法器

常用的 MSI 加法器有 7482（二位）、7483（四位）等。7483 为四位二进制加法器，有三个输入：四位加数 A、四位被加数 B 和低位进位 C_0，输出为四位和 S 及向高位的进位 C_4。

3.2.4.4 加法器的应用

3.2.4.4.1 代码转换

用 7483 可以把 8421BCD 码转换为余 3 码。余 3 码等于 8421BCD 码加 3，因此把 BCD 码接到 A 端，B 端接 0011（$B_1 = B_2 = 1$，$B_3 = B_4 = 0$），$C_0 = 0$，输出端 $S_1 \sim S_4$ 即为余 3 码。

3.2.4.4.2 四位二进制加/减器

减法可以用补码加法实现：

$$(R)_2 = (P)_2 - (Q)_2 = (P)_2 + \overline{(Q)}_2 + 1$$

用控制信号 S 选择加或减：$S = 0$ 时做加法，$S = 1$ 时做减法。把 Q 的每一位与 S 异或后接到 7483 的 B 端，并令 $C_0 = S$：

1. $S = 0$：B 端输入为 Q，$C_0 = 0$，实现 $P + Q$；
2. $S = 1$：B 端输入为 $\overline{Q}$，$C_0 = 1$，实现 $P + \overline{Q} + 1 = P - Q$。

3.2.4.4.3 二进制减法电路

设 N 为 n 位二进制数，则

$$N_{补} = 2^n - N_{原},\quad N_{补} = N_{反} + 1$$

所以

$$A - B = A - \left(2^n - B_{补}\right) = A + B_{反} + 1 - 2^n$$

实现时把 A 接 7483 的 A 端，B 取反后接 B 端，$C_0 = 1$。最高位的进位 C_4 反映了 A 与 B 的大小关系：有进位（$C_4 = 1$）时 $A \geq B$，结果为正，借位信号 $V = \overline{C_4} = 0$；无进位（$C_4 = 0$）时 $A < B$，结果为负，$V = 1$，此时输出是差值的补码，最高位为符号位。

例如（四位）：

$$0101 - 0001 = 0101 + 1110 + 1 = 1\,0100 \rightarrow (\text{舍去进位})0100$$

$$0001 - 0101 = 0001 + 1010 + 1 = 0\,1100$$

前者结果为 $+4$，后者 1100 是 -4 的补码。

3.2.4.4.4 8421BCD 码加法器

两个一位 8421BCD 码相加，和可能为 $0 \sim 18$。四位二进制加法器给出的是二进制和 S，当和大于 9 时必须加 6（0110）进行校正，才能得到正确的 BCD 码。

设第一次相加的和为 $S_4S_3S_2S_1$，进位为 C_4。需要校正的条件为

$$K_4 = C_4 + S_4S_3 + S_4S_2$$

即以下三种情况之一：$C_4=1$（对应 16 ~ 19）；$S_4=S_3=1$（对应 12 ~ 15）；$S_4=S_2=1$（对应 10、11、14、15）。

校正电路由两片 7483 组成：第一片完成 $A+B$，其输出 S 同时送到第二片的 A 端；第二片的 B 端接 0000 或 0110（由 K_4 控制），第二片的输出即为 BCD 码和，K_4 同时作为向高位的进位。

3.2.5 数值比较器

数值比较器用来判断两个二进制数的大小或是否相等。

3.2.5.1 一位数值比较器

设 A、B 为两个一位二进制数，比较结果用 $Y(A>B)$、$Y(A<B)$、$Y(A=B)$ 表示：

A	B	$Y(A>B)$	$Y(A<B)$	$Y(A=B)$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

表 3.18 一位数值比较器的真值表

$$Y(A>B)=A\overline{B},\quad Y(A<B)=\overline{A}B,\quad Y(A=B)=A\odot B$$

3.2.5.2 多位数值比较器

比较两个多位数时，应首先从高位开始逐位比较，高位能决定大小就不必再看低位。设

$$A=A_3A_2A_1A_0,\quad B=B_3B_2B_1B_0$$

则四位集成比较器 7485 的输出为

$$\begin{aligned}Y(A>B)=&A_3\overline{B}_3+(A_3\odot B_3)A_2\overline{B}_2+(A_3\odot B_3)(A_2\odot B_2)A_1\overline{B}_1\\&+(A_3\odot B_3)(A_2\odot B_2)(A_1\odot B_1)A_0\overline{B}_0\\&+(A_3\odot B_3)(A_2\odot B_2)(A_1\odot B_1)(A_0\odot B_0)I(A>B)\end{aligned}$$

$$Y(A<B)=\overline{A}_3B_3+(A_3\odot B_3)\overline{A}_2B_2+(A_3\odot B_3)(A_2\odot B_2)\overline{A}_1B_1$$
$$+(A_3\odot B_3)(A_2\odot B_2)(A_1\odot B_1)\overline{A}_0B_0$$
$$+(A_3\odot B_3)(A_2\odot B_2)(A_1\odot B_1)(A_0\odot B_0)I(A<B)$$
$$Y(A=B)=(A_3\odot B_3)(A_2\odot B_2)(A_1\odot B_1)(A_0\odot B_0)I(A=B)$$

其中 $I(A>B)$、$I(A<B)$、$I(A=B)$ 为级联输入，用来接收低位比较器的比较结果，以便扩展成更多位的比较器。单片使用时，应接 $I(A>B)=0$、$I(A<B)=0$、$I(A=B)=1$。

比较器的扩展有串行和并行两种接法：

1. **串行接法**：低位片的三个输出接到高位片的三个级联输入，高位片输出最终结果。电路简单，但位数越多，结果传递越慢。
2. **并行接法**：各片独立比较本段的四位，再用外部逻辑按“高位优先”的原则合并各片结果。电路复杂，但速度快。

小结

组合逻辑电路这一章围绕“分析”和“设计”两条主线展开：分析是由电路求功能，设计是由功能求电路。MSI 器件（编码器、译码器、数据选择器、加法器、数值比较器）则提供了模块化的实现手段，关键在于把待实现的逻辑函数变换成与器件功能表达式相对应的形式。

第二部分 时序逻辑电路

第四章 时序逻辑电路引论

定义 4.1 (时序逻辑电路)

电路在任何时刻的输出稳定值，不仅与该时刻的输入信号有关，而且与该时刻以前的电路状态有关；电路结构上具有反馈回路。

4.1 时序逻辑电路的基本概念

4.1.1 结构模型

时序逻辑电路一般由**组合电路**和**存储电路**两部分组成，其结构模型可以用四个信号来描述：

信号	名称	说明
X	外部输入信号	来自外部的输入变量，可以有多位
Z	外部输出信号	电路对外的输出变量，可以有多位
W	驱动信号	组合电路送给存储电路的激励信号
Q	状态信号	存储电路的输出，即电路当前的状态

表 4.1　时序逻辑电路的信号

描述时序逻辑电路的三组方程为

$$Z(t_n) = F[X(t_n), Q(t_n)] \quad \text{(输出方程)}$$

$$W(t_n) = G[X(t_n), Q(t_n)] \quad \text{(驱动方程)}$$

$$Q(t_{n+1}) = H[W(t_n), Q(t_n)] \quad \text{(状态方程)}$$

其中 $Q(t_{n+1})$ 称为电路的**次态**，$Q(t_n)$ 称为**现态**（或原状态）。

4.1.2 状态表和状态图

除三组方程外，时序电路还常用**状态表**和**状态图**来描述。

状态表把输入、现态、次态和输出列成表格，每一行对应一种“输入 + 现态”的组合；状态图则以圆圈表示状态，以带箭头的连线表示状态转换，箭头旁标注“输入/输出”：

$$Q^n \xrightarrow{X/Z} Q^{n+1}$$

例如，某时序电路有一个输入变量 x、两个状态变量 q_1 和 q_2、一个输出变量 z。两个状态变量的四种取值定义四个状态：

$$[q_1, q_2] = [0,0] \equiv A, \quad [0,1] \equiv B, \quad [1,0] \equiv C, \quad [1,1] \equiv D$$

设电路的状态表如下：

x	现态	次态	输出 z
0	A	D	0
1	A	C	1
0	B	B	1
1	B	A	0
0	C	C	1
1	C	D	0
0	D	A	0
1	D	B	1

表 4.2　某时序电路的状态表

由表 4.2 可画出状态图：A 在 $x=0$ 时转到 D 并输出 0，在 $x=1$ 时转到 C 并输出 1；其余状态依此类推。

当输入信号按 $0 \to 1 \to 1 \to 0 \to 1 \to 0 \to 1 \to 1 \to 0 \to 0$ 的顺序输入时，设电路初态为 A，则状态和输出的变化如下：

时刻	现态	输入	次态	输出
0	A	0	D	0
1	D	1	B	1
2	B	1	A	0
3	A	0	D	0
4	D	1	B	1
5	B	0	B	1
6	B	1	A	0
7	A	1	C	1
8	C	0	C	1
9	C	0	C	1

表 4.3 给定输入序列下的状态和输出变化

4.2 存储器件

存储电路由存储器件组成，能存储一位二值信号的器件称为**存储单元电路**。存储单元电路大多是**双稳态电路**。

双稳态电路的特点：

1. 具有两个稳定状态，用 0 和 1 表示。无外信号作用时，电路长期处于某个稳定状态，这两个稳定状态可用来表示一位二进制代码；
2. 有一个或多个输入端，在外加信号激励下，可使电路从一个状态转换成另一个状态。

存储单元电路分为两类：

1. **锁存器**：直接由激励信号控制电路状态的存储单元；
2. **触发器**：除激励信号外，还包含一个称为**时钟**的控制信号输入端，激励信号和时钟一起控制电路的状态。

4.3 锁存器

4.3.1 RS 锁存器

4.3.1.1 电路结构与逻辑符号

RS 锁存器由两个或非门交叉耦合而成，S_D 为置位端（置 1 端），R_D 为复位端（置 0 端）。定义 $Q=0$、$\overline{Q}=1$ 为 0 状态，$Q=1$、$\overline{Q}=0$ 为 1 状态。

4.3.1.2 逻辑功能分析

设电路的原状态为 Q^n，新状态为 Q^{n+1}。

（1）$S_D=0$、$R_D=0$（无激励信号）：两个或非门的输出保持原状态，

$$Q^{n+1}=Q^n$$

（2）$S_D=0$、$R_D=1$（置 0 信号有效）：

$$Q^{n+1}=0$$

（3）$S_D=1$、$R_D=0$（置 1 信号有效）：

$$Q^{n+1}=1$$

（4）$S_D=1$、$R_D=1$（置 0、置 1 同时有效）：两个或非门的输出同时为 0，破坏了两个输出应互补的关系。当两个激励信号同时消失后，电路的新状态取决于两个门延迟的微小差别，无法预先确定。因此一般情况下应禁止 $S_D=R_D=1$。

4.3.1.3 功能描述

RS 锁存器的特性表如下：

S_D	R_D	Q^{n+1}	功能
0	0	Q^n	保持
0	1	0	置 0
1	0	1	置 1
1	1	×	禁止

表 4.4 RS 锁存器的特性表

由特性表得特性方程

$$Q^{n+1}=S_D+\overline{R}_DQ^n,\quad S_DR_D=0$$

其中 $S_D R_D = 0$ 称为**约束条件**。

状态图如下：状态 0 在 $S_D = 1$、$R_D = 0$ 时转到状态 1；状态 1 在 $S_D = 0$、$R_D = 1$ 时转到状态 0；保持时 $S_D = 0$、$R_D = \times$（在状态 0）或 $S_D = \times$、$R_D = 0$（在状态 1）。

若用两个与非门交叉耦合构成 RS 锁存器，则输入为低电平有效，逻辑功能与或非门电路相同，只是激励信号的有效电平相反。

4.3.2 门控 RS 锁存器

在 RS 锁存器的基础上加控制信号 C，使锁存器状态转换的时间受 C 控制。令

$$R_D = R \cdot C, \quad S_D = S \cdot C$$

则：

1. 当 $C = 1$ 时，$R_D = R$、$S_D = S$，门控 RS 锁存器与 RS 锁存器功能完全相同；
2. 当 $C = 0$ 时，$R_D = S_D = 0$，锁存器状态保持不变。

门控 RS 锁存器的特性方程为

$$Q^{n+1} = S + \overline{R} Q^n, \quad SR = 0 \quad (C = 1 \text{ 时成立})$$

4.3.3 D 锁存器

D 锁存器能把呈现在激励输入端的单路数据 D 存入交叉耦合的锁存器单元中。它在门控 RS 锁存器的 S、R 端之间加一个反相器，使 S 与 R 始终相反，从而消除约束条件。

电路功能分析：

1. 当 $C = 0$ 时，$R_D = S_D = 1$（与非门电路），电路处于保持状态；
2. 当 $C = 1$ 时，$R_D = D$、$S_D = \overline{D}$，电路的新状态为 D。

D 锁存器的特性表为

D	Q^n	Q^{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

表 4.5 D 锁存器的特性表

特性方程为

$$Q^{n+1} = D$$

当 $C = 1$ 时，Q 跟随 D 变化，称为**透明**；当 $C = 0$ 时，Q 保持不变，称为**锁存**。

4.4 触发器

利用一个称为“时钟”的特殊定时控制信号去限制存储单元状态的改变时间，具有这种特点的存储单元电路称为**触发器**。

4.4.1 主从触发器

4.4.1.1 主从 RS 触发器

主从 RS 触发器由两个门控 RS 锁存器级联而成：主锁存器 F_1 和从锁存器 F_2 的时钟相位相反。

工作原理：

1. 当 $CP = 0$ 时，主锁存器 F_1 被选通，处于工作状态，按 S、R 的值改变中间状态 Q_m；从锁存器 F_2 被封锁，处于保持状态；
2. 当 $CP = 1$ 时，主锁存器 F_1 被封锁，进入保持状态；从锁存器 F_2 被选通，处于工作状态，输出 Q 根据 Q_m 的状态改变。

因此，CP 脉冲不论在低电平还是高电平期间，电路的输出状态最多只改变一次，不会出现**空翻**（控制信号有效期间输出状态发生多次变化的现象）。用于时序电路时，不会因不稳定而产生振荡。

电路符号中的“▷”称为**延迟符号**，表示该触发器在 $CP = 0$ 时接收 S、R 的数据，而在 CP 的上升沿时输出改变状态。

主从 RS 触发器的特性表和特性方程与 RS 锁存器基本相同，只是在列特性表时要加上 CP 脉冲标志：

$$Q^{n+1} = S + \overline{R}Q^n, \quad SR = 0 \quad (\text{仅在 } CP \text{ 有效沿到来时成立})$$

主从 RS 触发器仍有 $SR = 0$ 的约束。

4.4.1.2 主从 D 触发器

把主从 RS 触发器的主锁存器改成 D 锁存器，就得到主从 D 触发器：

1. 当 $CP=0$ 时，主锁存器被选通，$Q_m=D$，从锁存器保持原态；
2. 当 $CP=1$ 时，主锁存器保持原态，从锁存器被选通，$Q=Q_m$。

特性方程为

$$Q^{n+1}=D$$

输出状态在 CP 的上升沿改变。

4.4.1.3 主从 JK 触发器

主从 RS 触发器存在 $SR=0$ 的约束。为了消除这一约束，设计出主从 JK 触发器：以 D 触发器为核心，把输出反馈到输入端，使

$$D=J\overline{Q}^n+\overline{K}Q^n$$

因此

$$Q^{n+1}=D=J\overline{Q}^n+\overline{K}Q^n$$

电路以 D 触发器为核心，故不存在约束条件；CP 经一个非门送入 D 触发器，所以这种结构的 JK 触发器在 CP 下降沿到达时改变状态。

由特性方程容易求得特性表：

J	K	Q^{n+1}	功能
0	0	Q^n	保持
0	1	0	置 0
1	0	1	置 1
1	1	$\overline{Q}^n$	翻转

表 4.6 主从 JK 触发器的特性表

状态图：状态 0 在 $J=1$、$K=\times$ 时转到状态 1；状态 1 在 $J=\times$、$K=1$ 时转到状态 0；$J=0$、$K=\times$ 时状态 0 保持，$J=\times$、$K=0$ 时状态 1 保持，$J=K=1$ 时翻转。

带异步清零、置 1 端并具有多驱动输入端的 JK 触发器，其 J、K 端可以分别有多个输入：$J=J_1\cdot J_2$、$K=K_1\cdot K_2$。S_D 为异步置 1 端，R_D 为异步清零端。

主从触发器的抗干扰能力不强：在 CP 的某些区间内，输入端的干扰仍可能影响主锁存器的中间状态。为了提高抗干扰能力，应采用边沿触发器。

4.4.2 边沿触发器

边沿触发器的特点是：在时钟为稳定的 0 或 1 期间，输入信号都不能进入触发器，触发器的新状态仅决定于时钟脉冲有效边沿到达前一瞬间以及到达后极短一段时间内的输入信号。因此边沿触发器具有较好的抗干扰性能。

4.4.2.1 维持阻塞 D 触发器

维持阻塞 D 触发器由六个与非门组成，输出状态在 CP 的上升沿改变。

1. 当 $CP=0$ 时，内部节点 $M=N=1$，输出 Q 保持不变；
2. 当 CP 由 0 变为 1 时，电路按 CP 上升沿前一瞬间的 D 值改变输出，$Q^{n+1}=D$；
3. 在 $CP=1$ 期间，即使 D 发生变化，维持阻塞反馈线也会封锁输入，使输出保持不变；
4. S_D、R_D 为异步置 1、异步清零端，优先级高于时钟。

CP	S_D	R_D	D	Q^{n+1}
×	0	1	×	1
×	1	0	×	0
↑	1	1	0	0
↑	1	1	1	1

表 4.7 维持阻塞 D 触发器的功能表

4.4.2.2 负边沿 JK 触发器

负边沿 JK 触发器在 CP 的下降沿改变状态，其逻辑功能与主从 JK 触发器相同，但抗干扰能力更强。

4.4.2.3 CMOS 边沿 D 触发器

CMOS 边沿 D 触发器由 CMOS 传输门构成，属于主从结构，但具有边沿触发的特点：

1. 当 $CP=0$ 时，TG_1 和 TG_4 导通，TG_2 和 TG_3 截止，$Q'=D$，输出 Q 经 TG_4 回路保持原态；
2. 当 CP 由 0 变为 1 时，TG_2 和 TG_3 导通，TG_1 和 TG_4 截止，Q' 经 TG_2 保持 CP 上升沿到来前一瞬间的 D 信号，并经 TG_3 送到输出端。

因此它属于上升沿触发的 D 触发器。

4.4.3 集成触发器

常用的集成触发器有 74LS74（双 D 触发器，上升沿触发，带异步置位、清零端）、74LS112（双 JK 触发器，下降沿触发）和 74LS273（八 D 触发器，带公共清零端）等。使用时应注意：

1. 异步置位端 S_D、异步清零端 R_D 一般低电平有效，其作用优先于时钟；
2. 触发器正常工作时，S_D、R_D 应接无效电平；
3. 不同型号的触发器有效触发沿可能不同，画波形时必须先确认是上升沿还是下降沿触发。

4.5 触发器逻辑功能的转换

触发器逻辑功能的转换，是指用已有的触发器和适当的转换电路，实现另一种触发器的逻辑功能。设已有触发器的输入为 A、B，待求触发器的输入为 X、Y，则转换电路要实现的函数为

$$A = f_1(X, Y, Q^n), \quad B = f_2(X, Y, Q^n)$$

4.5.1 代数法

代数法通过比较已有触发器和待求触发器的特性方程，求转换电路的函数表达式。

4.5.1.1 JK 触发器转换为 D 触发器

已有 JK 触发器的特性方程为

$$Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$$

待求 D 触发器的特性方程为 $Q^{n+1} = D$。把 D 写成

$$D = D\left(Q^n + \overline{Q}^n\right) = D\overline{Q}^n + \overline{D}Q^n$$

比较两式得

$$J = D, \quad K = \overline{D}$$

4.5.1.2 JK 触发器转换为 T 触发器

T 触发器的功能为：$T = 0$ 时保持，$T = 1$ 时翻转。其特性方程为

$$Q^{n+1} = T\overline{Q}^n + \overline{T}Q^n$$

与 JK 触发器的特性方程比较得

$$J = K = T$$

当 $T = 1$ 时，触发器在 CP 控制下只具有翻转功能，称为 T' 触发器，此时 $J = K = 1$，$Q^{n+1} = \overline{Q}^n$。

4.5.1.3 RS 触发器转换为 D 触发器

RS 触发器的特性方程为

$$Q^{n+1} = S + \overline{R}Q^n, \quad SR = 0$$

把 D 触发器方程写成 $D = D + \overline{D}Q^n$，与上式比较得

$$S = D, \quad R = \overline{D}$$

此时 $SR = D\overline{D} = 0$，约束条件自然满足。

4.5.2 图表法

图表法先列出待求触发器的特性表，再根据已有触发器的特性，确定在每一种情况下已有触发器各输入端应加的信号，最后用卡诺图化简。

例如，把 RS 触发器转换为 JK 触发器：

J	K	Q^n	Q^{n+1}	S
R				
0	0	0	0	0
×	0	0	1	1
×	0	0	1	0
0	0	×	0	1
1	0	0	1	1
0	0	1	1	0
1	0	1	1	×
0	1	1	0	1
1	0	1	1	1
0	0	1		

表 4.8 RS 触发器转换为 JK 触发器的真值表

由卡诺图化简得

$$S = J\overline{Q}^n, \quad R = KQ^n$$

因此转换电路只需两个与门，把 J 与 $\overline{Q}^n$ 相与后接 S 端，把 K 与 Q^n 相与后接 R 端。

4.5.3 多功能触发器

例子 4.1

试用 D 触发器和四选一 MUX 构成一个多功能触发器，其功能如下表所示，表中 L、T 为控制变量，N 为数据输入变量。

L	T	N	Q^{n+1}
0	0	×	Q^n
0	1	×	Q^n
1	0	N	N
1	1	N	N

表 4.9 多功能触发器的功能表

以 L、T 作为 MUX 的地址变量，D 触发器的输入 D 就是 MUX 的输出。由功能表得

$$D_0 = Q^n, \quad D_1 = Q^n, \quad D_2 = N, \quad D_3 = N$$

解. 把 L、T 接到 MUX 的地址端，D_0、D_1 接 Q^n，D_2、D_3 接 N，MUX 的输出接 D 触发器的 D 端，即构成所需的多功能触发器。

4.6 触发器应用举例

4.6.1 消颤开关

机械开关在闭合或断开时，触点会发生多次抖动，产生一串窄脉冲。用 RS 锁存器可以构成消颤开关：两个按键分别把 S、R 置位和复位，锁存器的输出 Y 只取决于最后一次有效的置位或复位操作，触点的多次抖动不会使输出反复变化，从而得到干净的开关信号。

4.6.2 单脉冲发生器

用两个 D 触发器和一个与门可以构成单脉冲发生器。输入信号 d 接第一级触发器的 D 端，第一级的输出 Q_1 接第二级的 D 端，两级共用时钟 CP，输出为

$$Y = Q_1 \cdot \overline{Q_2}$$

当 $d = 1$ 时，第一个 CP 上升沿使 $Q_1 = 1$，此时 Q_2 仍为 0，故 $Y = 1$；第二个 CP 上升沿使 $Q_2 = 1$，于是 $Y = 0$。因此每按一次键，输出一个宽度等于一个时钟周期的单脉冲，脉冲宽度 $t_w = T_C$，与按键按下的时间长短无关。

第五章 时序逻辑电路的分析与设计

时序逻辑电路由组合电路和存储电路组成，其输出不仅与当前输入有关，还与电路当前的状态有关。描述时序电路的三组方程仍为

$$Z(t_n) = F[X(t_n), Q(t_n)] \quad \text{(输出方程)}$$

$$W(t_n) = G[X(t_n), Q(t_n)] \quad \text{(驱动方程)}$$

$$Q(t_{n+1}) = H[W(t_n), Q(t_n)] \quad \text{(状态方程)}$$

按存储单元状态改变的方式，时序电路分为**同步时序电路**和**异步时序电路**；按输出信号的特点，分为**米里（Mealy）型**和**摩尔（Moore）型**：

1. **米里型**：输出不仅取决于存储电路的状态，还取决于外部输入信号；
2. **摩尔型**：输出仅取决于存储电路的状态，与该时刻的外部输入无关。

5.1 MSI 构成的时序逻辑电路

5.1.1 寄存器和移位寄存器

5.1.1.1 寄存器

寄存器用于暂时存放二进制数码。一个 n 位寄存器由 n 个触发器组成，各触发器的时钟端并联，在同一个时钟脉冲作用下同时存入数据。

常用的集成寄存器有：

1. **74175**：四位 D 触发器寄存器，带公共时钟端和异步清零端 $\overline{R}_D$。$\overline{R}_D = 0$ 时输出全部清零；$\overline{R}_D = 1$ 时，在 CP 上升沿把 $d_1 \sim d_4$ 同时存入 $Q_1 \sim Q_4$。
2. **74173**：具有三态输出的四位缓冲数据寄存器，除时钟端和清零端外，还有两组输出使能端 G_1、G_2。当 G_1、G_2 均有效时输出数据，否则输出呈高阻态，便于与总线连接。

3. **74259**：八位可选址寄存器（地址锁存器），由地址 $A_2A_1A_0$ 选中八个输出中的一个，把数据 D 锁存到该位，其余位保持。它相当于一个数据分配器加八个锁存器。

5.1.1.2 移位寄存器

移位寄存器既能存放代码，又能使代码在时钟作用下逐位移动。按移位方向分为单向移位寄存器和双向移位寄存器；按输入、输出方式分为串入–串出、串入–并出、并入–串出、并入–并出四种。

5.1.1.2.1 单向移位寄存器

用四个 D 触发器可以组成串入–串/并出单向移位寄存器：输入数据 V_i 接第一级的 D 端，前一级的 Q 端接后一级的 D 端，各级共用时钟 CP。

设各触发器初态为 0，V_i 依次输入 $1 \to 0 \to 1 \to 1$，则每个时钟上升沿后寄存器的状态为

CP	Q_0	Q_1	Q_2	Q_3
初态	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	1	0	1	0
4	1	1	0	1

表 5.1 串入–串/并出移位寄存器的工作过程

连续输入四个 CP 脉冲后，在 Q_0、Q_1、Q_2、Q_3 端得到并行输出信号；若继续输入 CP 脉冲，可在串行输出端得到串行输出信号。

若要在串行输入的同时实现并行输入，可在每个触发器的 D 端加一个二选一电路：当接收控制信号 $S_D = 1$ 时，D 端接入并行数据；当 $S_D = 0$ 时，D 端接入前一级的 Q 端，实现移位。这样就构成串/并入–串出单向移位寄存器。

5.1.1.2.2 双向移位寄存器 74194

74194 是四位双向移位寄存器，具有异步清零、保持、右移、左移和并行置数五种功能。D_{SR} 为右移串行输入端，D_{SL} 为左移串行输入端，S_A、S_B 为工作方式控制端。

$\overline{R}_D$	S_A	S_B	CP	功能
0	×	×	×	异步清零
1	0	0	×	保持
1	0	1	↑	右移（$Q_0 \leftarrow D_{SR}$，$Q_i \leftarrow Q_{i-1}$）
1	1	0	↑	左移（$Q_3 \rightarrow D_{SL}$，$Q_i \rightarrow Q_{i+1}$）
1	1	1	↑	并行置数

表 5.2 74194 的功能表

注意：74194 的清零为异步，置数为同步。用两片 74194 可以接成八位双向移位寄存器，把低位片的 Q_3 接高位片的 D_{SR}，高位片的 Q_0 接低位片的 D_{SL}，两片的 S_A、S_B、CP 并联即可。

5.1.1.2.3 移位寄存器的应用

1. **可编程分频器**：用 74194 和 74138 构成。移位寄存器中只有一个 1 在循环，1 每移动一位，译码器就输出一个脉冲；改变译码器的输出接回置数端的组合，就可以改变分频比。
2. **串行加法器**：用两个 n 位移位寄存器分别存放加数 X 和被加数 Y，用一个 $n+1$ 位移位寄存器存放和，再配一个全加器和一个进位触发器。每个时钟周期从两个寄存器的最低位移出一位相加，和移入和寄存器，进位存入触发器供下一位使用。
3. **串行累加器**：用一个 n 位移位寄存器存放被加数，另一个 n 位移位寄存器存放累加和，全加器的进位经触发器反馈，实现串行累加。

5.1.2 计数器

计数器用于统计输入脉冲的个数。除直接计数外，还可用于定时、分频、程序控制和信号发生等。按计数方式分为同步计数器和异步计数器；按计数进制分为二进制计数器和非二进制计数器。

5.1.2.1 同步二进制加法计数器

用 T 触发器构成四位同步二进制加法计数器。设 $Q_3Q_2Q_1Q_0$ 为计数器的输出状态，Q_3 为高位，Q_0 为低位，C 为进位标志。各触发器的驱动方程为

$$T_0 = 1,\quad T_1 = Q_0^n,\quad T_2 = Q_1^n Q_0^n,\quad T_3 = Q_2^n Q_1^n Q_0^n$$

输出方程为

$$C = Q_3^n Q_2^n Q_1^n Q_0^n$$

由 T 触发器的特性方程 $Q^{n+1} = T \oplus Q^n$ 得状态方程

$$Q_0^{n+1} = \overline{Q_0^n}$$

$$Q_1^{n+1} = Q_1^n \oplus Q_0^n$$

$$Q_2^{n+1} = Q_2^n \oplus (Q_1^n Q_0^n)$$

$$Q_3^{n+1} = Q_3^n \oplus (Q_2^n Q_1^n Q_0^n)$$

计数顺序为 0000 → 0001 → 0002（即 0010）→ ⋯ → 1111 → 0000，每来一个 CP 脉冲加 1。

同步二进制加法计数器的特点：

1. 由 n 个触发器构成的计数器模为 2^n，没有多余状态，状态利用率最高；
2. 用 T 触发器构成时，电路结构有两条规则：$T_0 = 1$，$T_i = Q_{i-1}^n Q_{i-2}^n \cdots Q_0^n$（$i \neq 0$）；
3. 各触发器同时翻转，工作速度快，最高工作频率为

$$f_{\max} = \frac{1}{t_{PF} + t_{PG}}$$

其中 t_{PF} 为触发器的传输延迟时间，t_{PG} 为门电路的传输延迟时间。

5.1.2.2 集成同步二进制计数器 74161

74161 是四位同步二进制加法计数器，具有异步清零、同步置数、保持和计数功能，$\overline{R}_D$ 为异步清零端，$\overline{L}_D$ 为同步置数端，ENP、ENT 为两个使能端，CO 为进位输出端。

$\overline{R}_D$	$\overline{L}_D$	ENP	ENT	CP	功能
0	×	×	×	×	异步清零
1	0	×	×	↑	同步置数
1	1	0	×	×	保持
1	1	×	0	×	保持（$CO = 0$）
1	1	1	1	↑	同步计数

表 5.3 74161 的功能表

利用多片 74161 可以扩展计数器的位数：低位片的 CO 接高位片的 ENT，高位片的 $ENP=1$，各片 CP 并联，即可构成模 2^{4m} 的计数器。例如三片 74161 级联可构成模 2^{12} 计数器。也可以把后级芯片的 ENP、ENT 互换接法，构成不同速度的方案。

5.1.2.3 异步二进制计数器

用下降沿触发的 T' 触发器可以构成异步二进制计数器。把低位触发器的输出 Q_i 接到高位触发器的时钟端 CP_{i+1}，则构成加法计数器；若把 $\overline{Q}_i$ 接到 CP_{i+1}，则构成减法计数器。若采用上升沿触发器，则连接方式相反。

异步二进制计数器的特点：

1. 由 T' 触发器构成，触发器之间串接，低位触发器的输出作为高位触发器的时钟；
2. 触发器的状态翻转由低位向高位逐级进行，计数速度较低；
3. 若 CP 的频率为 f，则 Q_0、Q_1、Q_2、Q_3 输出脉冲的频率分别为 $\frac{f}{2}$、$\frac{f}{4}$、$\frac{f}{8}$、$\frac{f}{16}$，因此这种计数器也常作分频器使用。

5.1.2.4 二进制可逆计数器

可逆计数器有两种形式：

1. **有加/减控制的可逆计数器**：只有一个 CP 输入端和一个加/减控制端 U/D，由 U/D 决定计数方向；
2. **双时钟可逆计数器**：有两个 CP 输入端，加计数和减计数分别从不同的 CP 端输入。

用 T 触发器设计有加/减控制的同步二进制可逆计数器时，$T_0=1$，其余各位的驱动信号由 U/D 选择：

$$T_i = Q_{i-1}^n Q_{i-2}^n \cdots Q_0^n \quad (\mathrm{U/D}=1, \text{加计数})$$

$$T_i = \overline{Q_{i-1}^n Q_{i-2}^n} \cdots \overline{Q_0^n} \quad (\mathrm{U/D}=0, \text{减计数})$$

5.1.2.5 同步十进制计数器

以 T 触发器构成 8421BCD 码加法计数器为例。计数器的状态表如下：

Q_3^n	Q_2^n	Q_1^n	Q_0^n	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	C
0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	1	0	0
0	0	1	0	0	0	1	1	0
0	0	1	1	0	1	0	0	0
0	1	0	0	0	1	0	1	0
0	1	0	1	0	1	1	0	0
0	1	1	0	0	1	1	1	0
0	1	1	1	1	0	0	0	0
1	0	0	0	1	0	0	1	0
1	0	0	1	0	0	0	0	1
1	0	1	0	1	0	1	1	0
1	0	1	1	0	1	1	0	1
1	1	0	0	1	1	0	1	0
1	1	0	1	0	1	0	0	1
1	1	1	0	1	1	1	1	0
1	1	1	1	0	0	1	0	1

表 5.4 同步十进制加法计数器的状态表

由状态表求得各触发器的驱动方程和输出方程：

$$T_0 = 1$$

$$T_1 = \overline{Q_3^n} Q_0^n$$

$$T_2 = Q_1^n Q_0^n$$

$$T_3 = Q_2^n Q_1^n Q_0^n + Q_3^n Q_0^n$$

$$C = Q_3^n Q_0^n$$

状态方程为

$$Q_0^{n+1} = \overline{Q_0^n}$$

$$Q_1^{n+1} = Q_1^n \oplus \left(\overline{Q_3^n} Q_0^n\right)$$

$$Q_2^{n+1} = Q_2^n \oplus (Q_1^n Q_0^n)$$

$$Q_3^{n+1} = Q_3^n \oplus (Q_2^n Q_1^n Q_0^n + Q_3^n Q_0^n)$$

状态表中的后六个状态 $1010 \sim 1111$ 为无效状态。由状态方程可知：

$$1010 \to 1011 \to 0110, \quad 1100 \to 1101 \to 0100, \quad 1110 \to 1111 \to 0010$$

它们经过若干个时钟脉冲后都能回到有效循环，因此该电路具有**自启动特性**。

定义 5.1 (自启动特性)

时序电路由于某种原因进入无效状态后，若在若干个时钟脉冲作用下能自行返回到某个有效状态，进入有效循环，则称该电路具有自启动特性。

5.1.2.6 集成同步十进制计数器 74160

74160 是中规模集成同步十进制加法计数器，其逻辑符号、功能表和引脚排列与 74161 类同，只是计数模为 10，进位输出 CO 在计数到 9 时有效。利用 $\overline{R}_D$ 和 $\overline{L}_D$ 可以构成任意进制的计数器。

5.1.2.7 异步十进制计数器与 74290

异步十进制计数器由四个 T' 触发器组成，其中 Q_0 为模 2 计数，$Q_1Q_2Q_3$ 为模 5 计数，通过反馈使电路在 1001 后回到 0000。

74290 是二—五—十进制计数器，内部有两组彼此独立的计数器：一组为模 2 计数器（输出 Q_0），另一组为模 5 计数器（输出 $Q_1Q_2Q_3$）。通过外部连接可以构成十进制计数器：

1. $R_0(1) = R_0(2) = 1$ 时异步清零，输出全为 0；
2. $S_9(1) = S_9(2) = 1$ 时异步置 9，输出 $Q_3Q_2Q_1Q_0 = 1001$；
3. 将 Q_0 接 CP_1，计数脉冲从 CP_0 输入，输出为 $Q_3Q_2Q_1Q_0$，构成 8421BCD 码计数器；
4. 将 Q_3 接 CP_0，计数脉冲从 CP_1 输入，输出为 $Q_0Q_3Q_2Q_1$，构成 5421BCD 码计数器。

把两片 74290 级联，前一片的 Q_3 接后一片的 CP_0，即可构成模 100 的两位十进制计数器。

5.1.2.8 任意进制计数器

利用已有的中规模集成计数器，经外电路的不同连接，可以得到所需的任意进制计数器。常用方法有反馈复位法和反馈置位法。

5.1.2.8.1 反馈复位法

反馈复位法通过控制异步清零端 $\overline{R}_D$ 获得任意进制计数器。当计数器计到某个状态时，用门电路产生清零信号，使计数器立即回到 0000。

例如用 74160 构成模 6 加法计数器：计数器正常计数到 0110 时，令

$$\overline{R}_D = \overline{Q_2 Q_1}$$

则 $Q_2 = Q_1 = 1$ 时 $\overline{R}_D = 0$，计数器异步清零，有效状态为 0000 ~ 0101，共 6 个状态。

又如用 74161 构成模 10 计数器：计数到 1010 时清零，令

$$\overline{R}_D = \overline{Q_3 Q_1}$$

有效状态为 0000 ~ 1001。

反馈复位法的缺点是：

1. 存在一个极短的过渡状态（如 0110、1010），使输出产生毛刺；
2. 清零脉冲很窄，清零的可靠性较差。

提高清零可靠性的改进电路是在清零端加一个基本 RS 锁存器或门控电路，使清零信号在 $CP = 1$ 期间保持不变，直到 CP 回到低电平后再释放。

5.1.2.8.2 反馈置位法

反馈置位法利用计数器的同步预置数端 $\overline{L}_D$ 获得任意进制计数器。当计数器计到某个状态时，产生置数信号，在下一个时钟脉冲把预置数装入计数器。

例如用 74161 构成模 10 计数器：计数到 1001 时产生置数信号，令

$$\overline{L}_D = \overline{Q_3 Q_0}, \quad D_3 D_2 D_1 D_0 = 0000$$

则状态 0000 ~ 1001 循环，共 10 个状态。也可以改为计到 1111 时置入 0110，使状态在 0110 ~ 1111 之间循环，同样是模 10 计数器。

用 74161 构成 5421BCD 码计数器：计数到 1100 时置入 0000，令

$$\overline{L}_D = \overline{Q_3 Q_2}, \quad D_3 D_2 D_1 D_0 = 0000$$

则状态为 0000 ~ 0100、1000 ~ 1100，共 10 个状态，输出为 5421BCD 码。

利用置数法还可以构成可控模 10 计数器：当控制信号 $X = 1$ 时按 5421BCD 码计数，$X = 0$ 时按 8421BCD 码计数。方法是 $X = 1$ 时在 1100 置入 0000，$X = 0$ 时在 1010 复位清零，两种控制信号分别接到置数端和清零端。

用两片 74161 可以构成模 100 同步计数器。模 100 计数器从 0 计到 99，而 $99 = (01100011)_2$。当高位片、低位片的状态为 01100011 时产生置数信号：

$$\overline{L}_D = \overline{Q_6 Q_5 Q_1 Q_0}$$

在下一个时钟把两片都置入 0000，即得到模 100 计数器。

5.1.3 移位寄存器型计数器

移位寄存器型计数器是在移位寄存器的基础上加反馈电路而构成的具有特殊编码的同步计数器。除第一级外，其余各级满足

$$Q_i^{n+1} = Q_{i-1}^n$$

5.1.3.1 环形计数器

把移位寄存器的串行输出端和串行输入端相连，即 $D_0 = Q_3$，就构成环形计数器。

以四位环形计数器为例，有效循环为

$$1000 \to 0100 \to 0010 \to 0001 \to 1000$$

n 个触发器构成的环形计数器只有 n 个有效状态，有 $2^n - n$ 个无效状态，状态利用效率低。

实现自启动的方法有两种：利用触发器的置位、复位端把电路预置成有效循环中的某一状态；或者重新设计反馈电路。设反馈函数为 f，要求电路进入无效状态后能自动回到有效循环。对四位环形计数器，一种可行的反馈函数为

$$f = \overline{Q_0^n Q_1^n Q_2^n}$$

即 $Q_0 = Q_1 = Q_2 = 0$ 时反馈为 1（对应 0000、0001 两种状态），使 $0000 \to 1000$、$0001 \to 1000$；其余无效状态在移位过程中把多余的 1 逐位移出，最终进入单 1 的有效循环。

用 74194 构成环形计数器时，可令 $D_{SR} = Q_3 + \overline{Q_0 Q_1 Q_2}$。若输出全为 0，则通过 D_{SR} 移入 1，进入有效循环；否则经过移位总能把 1 移到 Q_3 处，电路进入置数状态，置入 1000，进入有效循环。

环形计数器的特点：

1. 附带有译码器功能，每个输出端的状态就是一个顺序脉冲；
2. 输出波形为顺序脉冲，常称为顺序脉冲发生器；

3. 缺点是状态利用效率低。

5.1.3.2 扭环形计数器

把移位寄存器的串行输出端取反后接回串行输入端，即 $D_0 = \overline{Q_3}$，就构成扭环形计数器。

四位扭环形计数器的有效循环为

$$0000 \to 1000 \to 1100 \to 1110 \to 1111 \to 0111 \to 0011 \to 0001 \to 0000$$

n 个触发器构成的扭环形计数器有 $2n$ 个有效状态，有 $2^n - 2n$ 个无效状态，状态利用效率比环形计数器高。

扭环形计数器的一种自启动反馈函数为

$$D_0 = \overline{Q_3^n} + Q_0^n \overline{Q_2^n}$$

它是在原反馈 $D_0 = \overline{Q_3^n}$ 的基础上，对部分无效状态修改反馈值，使无效状态经过若干次移位后进入有效循环。用 74194 构成时，令 $D_{SR} = \overline{Q_3} + Q_0 \overline{Q_2}$ 即可。

扭环形计数器的特点：

1. 输出码为循环码，相邻状态只有一位不同，能有效防止冒险现象；
2. 状态利用效率比环形计数器高，但仍低于二进制计数器。

5.1.4 序列信号发生器

序列信号发生器用于产生一串周期性的二进制序列，分为计数型和移存型两类。

5.1.4.1 计数型序列信号发生器

计数型序列信号发生器由计数器加组合电路组成。设计步骤：

1. 根据序列信号的长度 M 设计模 M 计数器；
2. 把计数器的输出作为输入，序列信号作为输出，列出真值表；
3. 由真值表求出组合逻辑表达式；
4. 画出逻辑图。

例子 5.1

试设计一个能产生序列信号 0101101 的计数型序列信号发生器。

序列长度为 7，用 74161 通过置数法构成模 7 计数器。把计数器的输出 $Q_2Q_1Q_0$ 作为输入，列出真值表：

Q_2	Q_1	Q_0	Z
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1

表 5.5　序列信号 0101101 的真值表

由卡诺图化简（$Q_2Q_1 = 11$ 为无关项）得

$$Z = Q_2 \oplus Q_0$$

解. 用模 7 计数器的 Q_2、Q_0 接异或门即可得到所需序列。计数器的状态按 $000 \to 001 \to \cdots \to 110$ 循环，输出端依次得到 0101101。

5.1.4.2 移存型序列信号发生器

移存型序列信号发生器由移位寄存器和组合反馈电路组成。设计时使移位寄存器型计数器的模等于序列长度，并让串行输入信号与所需序列一致。

例子 5.2

试设计一个能产生序列信号 00011101 的移存型序列信号发生器。

序列长度为 8，用 3 位移位寄存器（选用 74194，仅使用 Q_0、Q_1、Q_2）。把序列按 3 位划分：

$$000, 100, 110, 111, 011, 101, 010, 001$$

对应的状态为 $S_1 = 000$、$S_2 = 100$、$S_3 = 110$、$S_4 = 111$、$S_5 = 011$、$S_6 = 101$、$S_7 = 010$、$S_8 = 001$。输出取自 Q_2。

用四选一 MUX 实现反馈电路，取 Q_1、Q_2 为地址变量，由状态转换关系得

$$D_0 = 1, \quad D_1 = 0, \quad D_2 = Q_0^n, \quad D_3 = \overline{Q_0}^n$$

解. 把 MUX 的输出接 74194 的右移串行输入端 D_{SR}，Q_1、Q_2 接 MUX 的地址端，电路在时钟作用下循环产生 00011101。

说明

移存型序列信号发生器的状态划分不是唯一的。对同一序列，可以按不同的位数划分，得到不同的状态数和反馈函数。例如长度 5 的序列 10110 可按 4 位划分，用 74194 左移实现，D_{SL} 由 4 位状态的组合逻辑产生；位数取得越多，反馈函数越简单，但需要更多的触发器，实际设计时要在两者之间权衡。

5.2 同步时序逻辑电路的分析方法

分析的目的，是由给定的时序电路找出电路的逻辑功能。对时序电路而言，本质上是求电路在不同的外部输入和当前状态条件下的输出情况和状态转换规律。

由于同步时序电路中各触发器的动作在 CP 脉冲作用下同时发生，只要知道了当前状态下各触发器的输入（驱动信号），就能根据触发器的特性方程求得电路的次态。

分析步骤：

1. 列出时序电路的输出方程和驱动方程；
2. 将驱动方程代入触发器的特性方程，导出电路的状态方程；
3. 根据状态方程和输出方程列出状态表；
4. 根据状态表画出状态图或时序图；
5. 由状态表、状态图或时序图说明电路的逻辑功能。

例子 5.3

分析下图所示时序电路的逻辑功能。

电路由一级 JK 触发器和少量门电路组成，A、B 为外部输入，Z 为输出。由电路写出输出方程和驱动方程：

$$Z = A \oplus B \oplus Q^n$$

$$J = AB, \quad K = \overline{A + B}$$

代入 JK 触发器的特性方程 $Q^{n+1}=J\overline{Q}^n+\overline{K}Q^n$，得状态方程

$$Q^{n+1}=AB\overline{Q}^n+(A+B)Q^n$$

$$=AB+(A+B)Q^n$$

列出状态表：

A	B	Q^n	Q^{n+1}	Z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

表 5.6　全加器的状态表

解. 由状态表可见，$Z=A\oplus B\oplus Q^n$ 是本位和，Q^{n+1} 是本位向高位的进位。因此该电路是一个串行输入、串行输出的时序全加器：A、B 为两个二进制加数，Q^n 为低位来的进位，Z 为相加结果，Q^{n+1} 为向高位的进位。

例子 5.4

分析下列时序电路的逻辑功能。

电路由两级 JK 触发器组成，X 为输入，Z 为输出，状态变量为 Q_1Q_0。由电路可写出输出方程

$$Z=XQ_1^nQ_0^n$$

将驱动方程代入 JK 触发器特性方程，整理得状态方程

$$Q_0^{n+1}=X(Q_0^n+Q_1^n)$$

$$Q_1^{n+1}=X\left(\overline{Q_1}^n+Q_0^n\right)$$

列出状态表：

X	Q_1^n	Q_0^n	Q_1^{n+1}	Q_0^{n+1}	Z
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	0	1	0
1	1	1	1	1	1

表 5.7　1111 序列检测器的状态表

解. 由状态表可见，电路在连续输入四个或四个以上 1 时输出 1，因此是一个 1111 序列检测器。状态 $Q_1Q_0 = 11$ 表示已收到三个 1，此时若再输入 1，则输出 1 并保持在该状态。

5.3 同步时序逻辑电路的设计

同步时序逻辑电路的一般设计步骤：

1. 根据逻辑要求，建立原始状态表或原始状态图；
2. 利用状态化简技术简化原始状态表，消去多余状态；
3. 状态分配（状态编码），即将简化后的状态用二进制代码表示；
4. 选择触发器类型，并根据编码后的状态表求出驱动方程和输出方程；
5. 检查自启动性，若存在无效状态，必须检查电路能否自启动，不能自启动则需修改设计；
6. 画出逻辑图。

例子 5.5

用 D 触发器设计满足下列状态表所示的同步电路。表中 S 为状态，共有 A、B、C、D 四个状态；X 为输入变量。

S	$X=0$	$X=1$
A	A/0	B/0
B	A/0	C/1
C	B/0	D/0
D	C/1	D/0

表 5.8 原始状态表

四个状态需要两个触发器，取状态编码

$$A=00,\quad B=01,\quad C=11,\quad D=10$$

其中 Q_1Q_2 为状态变量。由编码后的状态表作卡诺图化简，得输出方程和驱动方程

$$Z=\overline{X}Q_1^n\overline{Q_2}^n+X\overline{Q_1}^nQ_2^n$$

$$D_1=Q_1^n\overline{Q_2}^n+XQ_2^n$$

$$D_2=X\oplus Q_1^n$$

解. 由于采用 D 触发器，D_1、D_2 就是加到两个触发器 D 端的驱动信号。按上式连接门电路和触发器，即可实现所需的状态转换和输出。该电路没有无效状态，不需要检查自启动性。

例子 5.6

试设计一个“111”序列检测器。要求：当连续输入三个或三个以上 1 时输出为 1，否则输出为 0。

设输入序列为 X，输出为 Z。建立原始状态表：

S	$X=0$	$X=1$
S_0	$S_0/0$	$S_1/0$
S_1	$S_0/0$	$S_2/0$
S_2	$S_0/0$	$S_3/1$
S_3	$S_0/0$	$S_3/1$

表 5.9 111 序列检测器的原始状态表

其中 S_0 为未收到 1 的状态，S_1 为收到一个 1 的状态，S_2 为连续收到两个 1 的状态，S_3 为连续收到三个或三个以上 1 的状态。

S_2 和 S_3 在相同输入下有相同的输出和相同的次态，因而是等价状态，可以合并，只保留 S_2。简化后剩三个状态，用两个触发器，取编码

$$S_0 = 00, \quad S_1 = 01, \quad S_2 = 10$$

11 为无效状态。由编码后的状态表作卡诺图化简，得

$$D_1 = X(Q_0^n + Q_1^n)$$

$$D_0 = X\overline{Q_1}^n\overline{Q_0}^n$$

$$Z = XQ_1^n$$

解. 检查自启动：当 $Q_1Q_0 = 11$ 时，若 $X = 0$，则 $D_1D_0 = 00$，次态为 00；若 $X = 1$，则 $D_1D_0 = 10$，次态为 10。两种情况下都能回到有效状态，因此电路能自启动。

例子 5.7

试设计一个模 4 可逆计数器。X 为控制信号，当 $X = 0$ 时做加计数，当 $X = 1$ 时做减计数，用 JK 触发器实现。

电路有四个状态 S_0、S_1、S_2、S_3，状态转换为：

1. $X = 0$ 时：$S_0 \to S_1 \to S_2 \to S_3 \to S_0$；
2. $X = 1$ 时：$S_0 \to S_3 \to S_2 \to S_1 \to S_0$。

取编码 $S_0 = 00$、$S_1 = 01$、$S_2 = 10$、$S_3 = 11$，得编码后的状态表。由状态表求得

$$Q_1^{n+1} = (Q_0^n \oplus X) \oplus Q_1^n$$

$$Q_0^{n+1} = \overline{Q_0^n}$$

与 JK 触发器的特性方程 $Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$ 比较，并利用 T 触发器的形式 $Q^{n+1} = T \oplus Q^n$，得

$$J_1 = K_1 = Q_0^n \oplus X$$

$$J_0 = K_0 = 1$$

解. 按上式连接两个 JK 触发器：J_1、K_1 接异或门（输入为 Q_0 和 X），J_0、K_0 接高电平 1，即可实现模 4 可逆计数。四个状态全部有效，不存在自启动问题。